



上海大学

SHANGHAI UNIVERSITY

本科毕业论文（设计）

UNDERGRADUATE THESIS (PROJECT)

题 目： 基于微服务架构的智能物流管理系统

学 院： 计算机科学与工程学院

专 业： 网络空间安全

学 号： 21122119

学生姓名： 付苗

指导教师： 牛志华

起讫日期： 2025 年 2 月 20 日 起 6 月 2 日止



姓 名：付苗

学号：21122119

论文题目：

原创性声明

本人声明：所呈交的论文是本人在指导教师指导下进行的研究工作。除了文中特别加以标注和致谢的地方外，论文中不包含其他人已发表或撰写过的研究成果。参与同一工作的其他同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

签 名：_____日 期：_____

本论文使用授权说明

本人完全了解上海大学有关保留、使用学位论文的规定，即：学校有权保留论文及送交论文复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容。

（保密的论文在解密后应遵守此规定）

签 名：_____指导教师签名：_____日期：_____

摘 要

随着电子商务、同城零售与跨区域配送需求的持续增长，传统物流管理系统在业务协同、资源调度、路径规划与运力分配等方面逐渐暴露出响应迟缓、耦合度高和扩展困难等问题。为提升物流系统在复杂业务场景下的可维护性、可扩展性与智能决策能力，本文设计并实现了一套基于微服务架构的智能物流管理系统。系统围绕订单履约、仓储库存、司机配送、路径规划与全国运力调度等核心业务展开，旨在构建兼具工程可落地性与算法智能性的现代物流平台。

本文首先分析了智能物流平台在顾客下单、商户发货、仓库分配、干线运输、末端配送和中转协同等环节的业务需求，并在此基础上采用微服务架构对系统进行拆分，形成认证服务、用户服务、顾客服务、商店与库存服务、订单服务、司机服务、物流规划服务和网关服务等多个独立服务单元。各服务通过 Spring Cloud Alibaba 生态完成服务注册发现、网关统一接入与服务间调用，并结合 RabbitMQ 实现跨服务异步解耦，结合 Redis 提升高频访问场景下的系统性能。

在技术实现上，后端采用 Spring Boot 3、Spring Cloud 2022 与 Spring Cloud Alibaba 构建微服务体系，数据访问层基于 MyBatis-Plus 与 MySQL 实现，缓存层采用 Redis，认证机制采用基于 BCrypt 与 JWT 的统一身份认证方案。前端采用 Vue3 + Vite 构建多角色单页应用，结合 Element Plus、Pinia 与 Leaflet 实现后台管理、状态管理与地图可视化。系统部署方面通过 Docker Compose 对数据库、中间件、微服务与前端容器进行统一编排。

在系统实现中，本文重点完成了以下内容：其一，基于真实角色权限建立了统一认证与网关鉴权机制，实现用户身份校验和路径级访问控制；其二，围绕订单、库存和配送任务构建了面向履约链路的业务协同流程，并采用消息队列实现库存扣减、配送创建与调度入池的最终一致性；其三，在物流规划服务中集成 GraphHopper 路网引擎，并实现了基于 A^* 的路径规划、基于大语言模型的候选路线决策、基于 K-Means 的末端调度预览以及基于最小费用最大流的全国运力规划；其四，构建了基于 Hub-and-Spoke 物流网络的完整履约流程，实现订单从下单、分配中转站、干线运输到末端派送的全链路闭环。

本文的创新点主要体现在三个方面：一是以微服务架构支撑复杂物流业务协同，实现了订单、仓储、配送与调度能力的服务化解耦；二是将真实路网路径规划、聚类调度与全国运力优化算法引入统一物流平台，增强了系统的智能化水平；三是引入大语言

模型对路线选择、批次建议与运力规划结果进行辅助分析，形成“确定性算法求解 + 生成式模型解释与裁决”的混合决策模式。实验与系统运行结果表明，该系统能够较好支撑智能物流管理的核心业务需求，并为微服务架构在智能物流领域的应用提供了可参考的工程实践方案。

关键词：微服务架构；Spring Cloud Alibaba；路径规划算法；大语言模型；Hub-and-Spoke 物流网络

ABSTRACT

With the rapid growth of e-commerce and regional distribution demand, traditional logistics systems increasingly suffer from poor scalability, tight coupling, and insufficient decision support in routing and capacity coordination. To address these issues, this thesis designs and implements an intelligent logistics management system based on microservice architecture. The system focuses on order fulfillment, inventory collaboration, driver delivery, route planning, and nationwide transportation scheduling, aiming to build a modern logistics platform with both engineering practicability and intelligent optimization capability.

This work first analyzes the business requirements of an intelligent logistics platform, including customer ordering, merchant shipment, warehouse allocation, trunk transportation, hub transfer, and last-mile delivery. Based on these requirements, the system is decomposed into multiple independent services such as authentication service, user service, customer service, shop and inventory service, order service, driver service, logistics service, and gateway service. Service registration, discovery, unified gateway access, and inter-service communication are built on the Spring Cloud Alibaba ecosystem, while RabbitMQ and Redis are used for asynchronous decoupling and high-frequency access acceleration.

For implementation, the backend adopts Spring Boot, Spring Cloud, and Spring Cloud Alibaba to construct the microservice architecture. The data access layer is implemented with MyBatis-Plus and MySQL, while Redis is used as the cache layer. User authentication is based on BCrypt password hashing and JWT token issuance. The frontend is built with Vue3 and Vite as a multi-role single-page application, together with Element Plus, Pinia, and Leaflet for interface rendering, state management, and map visualization. The whole system is containerized and orchestrated through Docker Compose.

The system implementation mainly covers four aspects. First, a unified authentication and gateway authorization mechanism is established to support identity validation and path-level access control. Second, the fulfillment chain for orders, inventory, and delivery tasks is coordinated across services, and eventual consistency is ensured through message queues. Third, the logistics service integrates GraphHopper for real road network loading, implements A*-based route planning, large-language-model-assisted candidate route selection, K-Means-based dispatch preview, and minimum-cost maximum-flow based nationwide capacity planning. Fourth, a complete Hub-and-Spoke delivery workflow is built to support the full lifecycle from order creation and hub assignment to trunk

transportation and last-mile fulfillment.

The main contributions of this thesis are threefold. First, it demonstrates how microservice architecture can support complex logistics collaboration by decoupling order, warehouse, delivery, and scheduling capabilities. Second, it integrates route planning, clustering-based dispatching, and nationwide capacity optimization algorithms into a unified logistics platform. Third, it introduces large language models to assist with route judgement, batch advice, and flow-plan interpretation, forming a hybrid decision-making mode that combines deterministic optimization with generative reasoning. The results show that the proposed system effectively supports the core requirements of intelligent logistics management and provides a practical reference for applying microservice architecture in this domain.

Keywords: Microservice Architecture; Spring Cloud Alibaba; Route Planning Algorithm; Large Language Model; Hub-and-Spoke Logistics Network

1 目录

摘 要	I
ABSTRACT.....	III
1 绪论	1
1.1 研究背景	1
1.2 研究意义	2
1.3 主要研究内容	3
1.4 各章内容与架构.....	3
2 相关技术综述	5
2.1 微服务架构相关技术	5
2.1.1 注册中心.....	5
2.1.2 微服务网关与服务调用.....	6
2.1.3 消息队列.....	6
2.2 后端其他技术	7
2.2.1 MySQL 与 R2DBC 响应式编程	7
2.2.2 Redis 缓存	8
2.3 前端技术栈	8
2.3.1 响应式框架	8
2.3.2 服务端渲染框架.....	9
2.4 容器化部署技术.....	9
2.4.1 Docker 容器	10
2.4.1 Docker Compose.....	10
2.4.1 Docker Swarm 与 Kubernetes	10
2.6 本章小结... 10	
3 系统设计	13
3.1 需求分析	13

3.1.1 总体分析.....	13
3.1.2 功能需求-个人用户用例	14
3.1.3 功能需求-商业用户用例	15
3.1.4 功能需求-司机用例.....	15
3.1.5 功能需求-仓库/网点用例.....	16
3.1.6 非功能需求	17
3.2 架构设计	17
3.3 后端设计	17
3.3.1 系统管理模块设计	18
3.3.2 个人模块设计	19
3.3.3 电商模块设计	20
3.3.4 司机模块设计	22
3.3.5 仓储模块设计	24
3.3.6 物流单模块设计.....	25
3.3.7 物流规划模块.....	26
3.4 前端设计	26
3.5 本章小结	31
4 系统实现	32
4.1 微服务环境搭建.....	32
4.2 认证与权限系统实现... 26	
4.3 核心业务服务实现... 26	
4.4 智能算法实现... 26	
4.5 前端实现... 26	
4.6 容器化部署实现... 26	
5 系统展示	45
5.1 并发性能测试... 27	
5.2 个人用户功能展示... 27	
5.3 商户用户功能展示... 27	
5.4 司机用户功能展示... 27	
5.5 管理员功能展示... 27	

结论 49

 6.1 总结..... 49

 6.2 不足与展望 50

参考文献 53

附录... 34

致谢... 35

a) 1 绪论

本章主要介绍论文的研究背景、研究意义、主要研究内容以及组织结构。首先讨论了物流行业的背景、智能物流行业使用微服务架构的原因及意义；接着，概述了微服务架构及相关技术的研究现状，包括其思想演进及技术栈分析；然后，明确了本文的研究内容，即基于微服务架构的智能物流管理系统；最后，简要介绍了论文的组织结构，包括各章的主要内容以及结构。

1.1 研究背景

物流行业作为支撑国民经济发展的基础性、战略性、先导性产业，在全球化和电子商务迅猛发展的浪潮下，其重要性日益凸显。然而，传统的物流管理模式在应对日益增长的业务复杂性、时效性要求以及成本控制压力时，逐渐显现出诸多瓶颈。当前物流行业普遍面临着高成本、低效率、信息孤岛等痛点。例如，中国的物流成本占 GDP 比重曾高达 14.1%，远高于许多发达国家的平均水平，同时仓库闲置率等资源浪费问题也较为突出。这些问题严重制约了物流行业的进一步发展和整体服务水平的提升。

为应对这些挑战，在物联网、大数据、人工智能以及云计算等新一代信息技术的驱动下，物流行业正经历着深刻的数字化和智能化转型[1]。智能物流管理系统应运而生，旨在通过技术创新优化物流运作的各个环节。物联网技术通过各类传感器和标识设备，实现了对货物、车辆、仓库等物流要素状态的实时感知与数据采集，有效打破了传统模式下的信息壁垒[2]。大数据技术则能够对海量的物流运作数据进行深度挖掘与分析，为路径优化、需求预测、库存管理等方面提供决策支持，显著提升运营效率[3]。人工智能技术，尤其在智能调度、风险预警、自动化处理等方面的应用，进一步提升了物流系统的自动化水平和智能化程度[4]。

然而，随着业务的扩展和深化，传统的单体式系统架构在支撑复杂的智能物流应用时，其固有的技术局限性愈发明显。单体架构系统往往功能模块耦合度高，任何微小的改动或升级都可能影响整个系统的稳定性，导致维护成本高、迭代周期长，难以快速响应市场变化和满足高并发、高可用的需求[5]。特别是在物流高峰期（如电商促销活动期间），单体系统容易因资源瓶颈而出现性能问题甚至服

务中断。因此，寻求一种更灵活、更具扩展性、更适应现代物流业务动态需求的系统架构成为业界的迫切需求。

在此背景下，微服务架构作为一种新兴的软件架构模式，受到了广泛关注。微服务架构倡导将复杂的系统拆分为一组小而自治的服务单元，每个服务单元围绕特定的业务能力构建，可以独立开发、部署、扩展和管理[6]。这种架构模式天然契合了智能物流系统对灵活性、可扩展性和高可用性的要求。通过采用微服务架构，可以将智能物流管理系统中的订单管理、库存管理、运输调度、配送跟踪、路径优化等核心功能模块解耦为独立的服务，从而提高系统的整体韧性、开发效率和技术选型的灵活性。

1.2 研究意义

基于上述背景，本研究旨在设计并实现一个基于微服务架构的智能物流管理系统。该系统的研究与实现具有重要的理论价值和现实意义：

1. 提高物流运作效率：通过微服务实现各业务模块的解耦和高效协同，结合路径优化、智能调度等功能，可以显著缩短运输时间，提高车辆、仓储等资源的利用率，从而全面提升物流运作效率。

2. 降低运营成本：微服务架构的弹性伸缩能力有助于根据业务负载动态调整资源，避免资源浪费。同时，优化的路径规划和库存管理也能有效降低运输成本和仓储成本。系统自动化水平的提高也能减少对人力的依赖，降低人力成本。

3. 提升用户体验：系统提供实时的订单跟踪、精准的预计送达时间以及便捷的查询服务，能够显著提升货主、承运方以及最终客户的使用体验和满意度。各服务独立部署和升级的特性也能保障核心业务的连续性，减少因系统维护带来的服务中断。

4. 促进物流行业数字化转型：本研究探索将先进的微服务架构应用于复杂的物流管理场景，为传统物流企业向数字化、智能化转型提供了可借鉴的技术方案 and 实践经验。系统的模块化设计也便于未来集成更多新兴技术（如区块链、数字孪生等），推动行业的持续创新。

5. 验证微服务架构优势：通过在智能物流领域的具体实践，可以深入验证微服务架构在应对高并发、复杂业务逻辑、快速迭代需求方面的优势和适用性，

为相关领域的系统架构选型提供参考。

1.3 主要研究内容

本课题的核心任务是设计、开发并实现一个功能相对完善、具备良好扩展性的基于微服务架构的智能物流管理系统。主要研究内容包括以下几个方面：

1. 系统需求分析与架构设计：深入分析智能物流管理的核心业务流程和功能需求，设计基于微服务理念的系统总体架构，划分核心服务模块（如订单服务、库存服务、调度服务、追踪服务、路径优化服务等），并定义服务间的交互接口与协议。

2. 关键技术选型与应用：研究并选用合适的技术栈来实现微服务架构，重点围绕 Spring Cloud 生态（如服务注册与发现、API 网关、配置中心、服务调用、熔断降级等）进行技术实践。同时，研究前端技术（如 Vue3, Vite）与后端微服务的交互模式。

3. 核心服务模块的设计与实现：针对订单管理、库存管理、运输调度、配送跟踪、路径优化等关键业务模块，进行详细的功能设计和数据库设计，并完成各微服务的编码实现。

4. 系统集成与部署：实现各微服务之间的集成与协同工作，并研究使用容器化技术和容器编排工具对系统进行打包、部署和管理，以实现弹性伸缩和高可用性。

5. 系统测试与评估：对系统进行功能测试、性能测试和可用性测试，评估系统是否满足设计要求，并分析其优势与待改进之处。

1.4 各章内容与架构

全文共分为五章正文、结论与附录，整体结构围绕“需求分析—系统设计—系统实现—系统展示—总结与展望”的逻辑展开。

第一章为绪论，主要介绍研究背景、研究意义、主要研究内容以及论文整体结构，说明选题来源与研究价值。

第二章为相关技术综述，重点介绍系统所采用的微服务治理技术、数据访问与缓存技术、前端开发技术、容器化部署技术，以及路径规划、聚类调度、最小费用

最大流和大语言模型等智能算法相关技术。

第三章为系统设计，围绕需求分析、总体架构设计、核心模块设计与前端设计展开，给出系统角色划分、服务划分、数据模型与主要接口设计方案。

第四章为系统实现，主要介绍系统的具体编码实现过程，包括微服务环境搭建、认证与权限系统实现、订单与库存等核心业务服务实现、智能路径规划与运力规划算法实现、前端多角色界面实现以及 Docker 容器化部署配置。

第五章为系统展示，主要通过界面截图、流程展示与性能测试结果说明系统在顾客、商户、司机与管理员多角色场景下的实际运行效果，并展示路径规划、末端调度和全国运力规划等功能模块的交互界面。

结论部分总结本文的主要研究工作与系统实现成果，分析系统当前存在的不足，并对后续在算法优化、部署架构升级与业务扩展等方面的发展方向进行展望。

b) 2 相关技术综述

本章围绕基于微服务架构的智能物流管理系统所采用的关键技术展开综述。内容主要包括微服务治理与服务通信技术、后端数据访问与缓存技术、前端开发与地图可视化技术、容器化部署技术，以及系统中涉及的路径规划、聚类调度、运力优化与大语言模型辅助决策等智能算法技术。这些技术共同构成了系统设计与实现的理论基础与工程支撑。

2.1 微服务架构相关技术

微服务架构是一种软件工程概念，主要强调在后端项目管理中实现高内聚、低耦合。在传统单体架构中，随着需求的不断引入，软件各个组件紧密耦合，管理成本越来越高。在微服务架构中，每个单位可以根据独立的需求设计、开发、测试、部署和维护，因而可以实现低成本的扩展和维护。

对于智能物流管理系统而言，其业务流程复杂、涉及模块众多，且对系统的可用性、可扩展性要求高。微服务架构能够很好地满足这些需求：将不同的物流功能拆分为独立服务，便于功能的快速迭代和优化；可以根据业务高峰独立扩展订单服务或调度服务；单个模块的升级或故障不会影响其他核心业务的运行。

为了实现微服务架构，首先需要对各个子模块进行精确划分与设计，这点将在下一章详细讨论。此外，微服务架构在实现中需要引入多种中间件，将各个部分有机地组合在一起。

2.1.1 注册中心

由于各微服务间可以自由选择是否启用、部署多少个镜像，我们需要在后台需要有一个注册中心组件提供服务发现、故障转移、负载均衡等功能。该中间件使用一个列表实时维护各个服务的可用性及访问方式，这些信息通过与微服务间的心跳包通信获取。常见的注册中心有 Zookeeper、Eureka、Nacos 等。

Zookeeper 是 Hadoop 的子项目，在分布式系统中提供管理集群、服务请求、数据订阅等功能。Zookeeper 在实现注册中心时会以路径的形式维护各个微服务。然而，Zookeeper 保障一致性、分区容忍性而不保障可用性，因此在故障发生时

可能导致系统服务中断。

Eureka 是 Spring Cloud 的组件之一，基于 Client/Server 架构提供注册中心服务，且将各个微服务的信息保存在 ConcurrentMap 中。Eureka 以镜像的方式实现了高可用，但其在设计上优先保证一致性和分区容忍性。Eureka 是当前最广泛应用的注册中心中间件，但 Eureka 不提供配置管理功能，因此需要结合 Github 搭建配置中心，无法在运行时修改相关配置。

Nacos 是阿里集团开发的一款轻量级注册中心服务。它配置简单，且具备动态修改服务配置的功能。Nacos 还支持在 CP 和 AP 模式间自动选择，从而提高了更好的性能。

由于 Nacos 的种种优势，我们在项目中使用 Nacos 作为注册中心。

2.1.2 微服务网关与服务调用

微服务架构中，由于服务间调用关系错综复杂，且权限管理需跨越多个服务，我们需要一个独立网关拦截并路由对各个 API 的调用请求。该组件可以利用注册中心发现的服务信息，实现服务调用的全局管理。此外，在微服务架构中，不仅存在客户端对微服务的请求，还存在微服务对微服务的请求，因此我们需要一个便捷方式在 SpringBoot 框架中实现对 HTTP 请求的封装。

Spring Cloud Gateway 是 Spring Cloud 官方推出的新一代 API 网关，基于 Spring WebFlux 和 Reactor 模型构建，能够提供统一入口、请求路由、过滤器链处理、身份认证与权限控制等能力。与早期的 Zuul 1.x 相比，Spring Cloud Gateway 在性能、可扩展性和与现代 Spring 生态集成方面更具优势，已成为当前微服务系统的主流网关方案。

OpenFeign 是声明式服务调用组件，开发者只需定义带注解的接口，即可完成对远程微服务的 HTTP 调用封装。在配合 Spring Cloud LoadBalancer 使用时，OpenFeign 可以自动完成基于服务名的实例发现与负载均衡，降低服务间调用的样板代码。

在本项目中，系统使用 Spring Cloud Gateway 作为统一入口，实现 JWT 认证过滤、角色路径权限控制以及用户身份透传；同时使用 OpenFeign 完成订单、库存、顾客与物流规划等服务之间的同步调用，形成完整的微服务协作链路。

2.1.3 消息队列

在微服务架构中，服务间的通信是关键环节。除了同步的 RESTful API 调用外，异步消息传递机制在许多场景下更为适用，而消息队列是实现异步通信的核心组件，它主要有服务解耦、异步通信、削峰填谷等作用。

使用消息队列时，各个服务不需要直接与其他服务进行紧密的代码或接口调用，只需要关注自己与消息队列的交互，有效降低了服务间的耦合度，使得服务可以更容易的独立开发和部署。此外，消息队列可以让调用方将消息发送到队列后，无需等待被调用方的响应即可继续执行后续操作，从而提高系统的吞吐量和响应速度。在业务高峰期时，会出现大量请求同时涌入的情况，此时便可以将请求先放入消息队列中，从而起到缓冲的作用，消费者服务按照自身处理能力进行消费，避免瞬时高并发冲垮下游服务，保证系统的稳定性。

当前常用的消息队列中间件有 RabbitMQ、Kafka 等。RabbitMQ 提供了灵活的路由和可靠的消息确认机制，适合业务逻辑复杂的场景。Kafka 则以高吞吐量和持久化日志著称，适合大数据流处理和日志收集。

在本项目中，RabbitMQ 扮演着核心消息中间件的角色，其在订单状态更新、库存变更通知、运输调度和位置信息更新等方面，通过强大的消息路由、可靠性保证和灵活的消息处理机制，为本系统提供了可靠的消息通信基础设施。

2.2 后端其他技术

除微服务治理组件外，后端系统还需要稳定的数据持久化、便捷的数据访问抽象与高效的缓存机制。为此，本项目采用 MySQL 作为主要关系型数据库，以 MyBatis-Plus 作为数据访问增强框架，并结合 Redis 提升热点数据读取效率。

2.2.1 MySQL 与 MyBatis-Plus

MySQL 是当前广泛使用的开源关系型数据库管理系统，具备成熟的事务支持、索引机制与较高的工程稳定性，适合支撑订单、库存、配送和调度等结构化业务数据存储需求。在本系统中，MySQL 8.0 主要用于保存用户、商品、仓库、订单、路线、批次以及全国运力规划结果等核心业务数据。

MyBatis-Plus 是构建在 MyBatis 之上的增强工具，在保留原生 SQL 控制能力的基础上，提供了通用 CRUD 接口、条件构造器、分页插件、自动填充等功能，

可显著减少样板代码。在本项目中，各服务普遍采用 `ServiceImpl<M,T>` 形式构建业务服务层，并通过 `LambdaQueryWrapper` 实现类型安全的条件查询，从而提升开发效率与可维护性。

此外，系统在运行时使用 `Spring Boot` 默认集成的 `HikariCP` 连接池来管理数据库连接，通过限制最大连接数、配置超时时间和复用空闲连接等方式，提高并发场景下的数据访问稳定性。相较于完全响应式的数据访问方案，本项目采用阻塞式的 `MyBatis-Plus + JDBC` 技术路径，更符合系统当前的业务实现方式和工程复杂度要求。

2.2.2 Redis 缓存

在高并发状态下，`MySQL` 由于基于块设备存储数据，速度远慢于内存，因此需要基于内存的 `NoSQL` 实现缓存加速。

`Redis` 是目前最主流的内存 `KV` 存储中间件，它支持多种数据类型的存储，同时具有极高性能的读写能力，并提供了 `RDB` 和 `AOF` 两种数据持久化技术和集群模式、哨兵模式等高可用方案。

基于 `Redis` 的特性，本项目使用 `Redis` 缓存访问频率较高的热点数据，以减轻后端数据库的负载。在收到订单消息时，我们将同时更新 `Redis` 缓存中的订单数据和搜索文档，并设置有 24 小时的缓存过期时间。此外，本项目使用 `Redis` 保存用户认证 `Token` 等会话信息，实现快速身份验证。

2.3 前端技术栈

用户交互界面的质量直接影响本系统的用户体验。现代前端开发普遍是基于 `Webpack`、`Vite` 等打包器的 `Typescript` 项目。这些项目使用 `Vue` 或 `React` 这样的响应式框架，通过 `axios`、`ofetch` 等 `ajax` 请求组件实现与后端的交互。

2.3.1 响应式框架

当今，前端开发主要基于 `MVC` 或 `MVVM` 的范式，将页面结构与注入的数据分离。前端代码主要关心数据的 `schema` 模型、获取方式，以及页面的组织结构、组织样式等。上述开发范式主要通过 `Vue`、`React`、`Angular` 等响应式框架实现。本项目规模较小，为了开发效率，选择轻量级的 `Vue` 进行前端开发。

Vue.js 是一个流行的、渐进式 JavaScript 框架，以其易学易用、性能优越和灵活性著称。本系统中所使用的 Vue3 版本通过优化的虚拟 DOM 算法、编译时优化等手段，提供了更好的运行时性能和更小的包体积，相较于先前的 Vue2 性能实现了优化。Vue3 还提供了 Composition API，这是一种更灵活、可组合的组织组件逻辑的方式，特别适合大型复杂组件的状态管理和逻辑复用，提高了代码的可维护性和可读性。Vue3 还有着更好的 TypeScript 支持，对 TypeScript 的集成更加友好，有助于构建类型安全、更健壮的前端应用。

2.3.2 构建工具与界面组件

Vite 是新一代前端构建工具，充分利用浏览器原生 ES Module 特性以获得更快的开发服务器启动速度，并在生产环境通过高效打包提升构建性能。对于本系统这类以后台管理和多角色交互为主的应用场景，采用 Vue3 + Vite 的单页应用模式即可满足开发效率与运行性能要求，无需引入以 SSR 为主要特征的框架。

Element Plus 是基于 Vue 3 的组件库，提供表格、表单、弹窗、分页、菜单等通用后台组件，可帮助开发者快速构建风格统一的管理界面。本项目的大部分后台页面均基于 Element Plus 实现，并结合 Pinia 进行前端状态管理，结合 axios 完成与后端接口的数据交互。

在地图与轨迹展示方面，系统前端采用 Leaflet 作为基础地图渲染组件，并结合后端返回的 GeoJSON 路线数据实现运输轨迹、司机位置和全国网络拓扑的可视化展示。这种组合方式既保证了开源可控性，也满足了物流地图场景下的交互需求。

2.4 容器化部署技术

容器是一个隔离的运行环境，具有独立性、可移植性、快速启动和资源利用率高的特点。在微服务架构下，各个模块间彼此独立、可分布式扩展。容器化部署技术即将这些模块各自打包为容器镜像，再利用统一的编排配置文件，根据客户端调用负载强度按需启用和扩容。

后端项目中微服务架构的设计与实现，仅仅在工程角度实现了该架构可扩展性的优点。我们还需要结合容器化部署编排技术，才能实现弹性扩容、故障迁移等功能，最大化微服务架构的效益。

2.4.1 Docker 容器

Docker 是目前最主流的开源容器化平台[30]。它利用 Linux 内核的特性，允许开发者将应用程序及其所有依赖打包到一个轻量级、可移植的独立容器镜像中。容器在运行时相互隔离，共享宿主机的操作系统内核。

前文中提到的各种中间件依赖可以直接使用标准化的官方容器，从而无需配置运行环境，且可便捷指定端口号和配置参数。本项目的前端、后端实现也会提供对应的 Dockerfile，从而在任何支持 Docker 的环境中可以标准化部署，不存在兼容性问题。

2.4.2 Docker Compose

Docker Compose 是一个用于定义和管理多容器 Docker 应用的工具，开发者可以通过一个 YAML 文件来配置应用所需的所有容器、网络、数据卷等，此后仅需一条命令即可启动、停止和重建所有服务。此外，Docker Compose 还可以为多个关联容器创建一个隔离的局域网，并可指定容器间的依赖关系。

本系统中的前端、后端、中间件之间依赖关系复杂，端口号配置繁多。在使用 Docker Compose 后，仅需提供一个全局配置文件即可让用户快速部署。

2.4.3 容器编排扩展讨论

对于毕业设计阶段的部署需求，Docker Compose 已能够满足数据库、中间件、微服务与前端应用的一键编排与统一启动要求，因此本文采用单机容器化部署方案。需要指出的是，在面向生产环境的大规模部署场景中，仍可进一步引入 Kubernetes 等成熟编排平台，以获得自动扩缩容、故障自愈和更强的运维能力，但这并非本文系统实现的重点。

当前，Docker Swarm 集成在了 Docker-CE Runtime 中。因此，在实践中，仅需在 Docker Compose 配置文件中指定相关的配置项，即可完成相应配置。

2.5 智能算法技术

除传统业务系统技术外，本项目还引入了面向物流优化场景的智能算法技术，包括基于真实路网的路径搜索、多候选路线裁决、聚类调度预览以及全国运力优化等。相关技术的引入使系统从传统信息管理平台进一步演进为具备辅助决策能力

的智能物流平台。

2.5.1 路网引擎（GraphHopper）

GraphHopper 是一款高性能的 Java 路径规划引擎，能够加载 OpenStreetMap 地图数据并构建可查询的道路图结构。与简单直线距离估算不同，GraphHopper 可以基于真实道路拓扑和车辆通行规则执行路径搜索，因此更适合物流运输场景中的实际路线规划。在本系统中，GraphHopper 被集成至 logistics-service，用于加载目标区域的 OSM 数据并为上层 A* 路径算法提供底层路网支持。

2.5.2 A* 启发式搜索算法

A* 算法是一种经典的启发式最优路径搜索算法，通过代价函数 $f(n)=g(n)+h(n)$ 引导搜索方向。其中 $g(n)$ 表示起点到当前节点的实际代价， $h(n)$ 表示当前节点到目标节点的启发式估计。在本系统中，启发函数采用 Haversine 球面距离，由于该距离不会高估真实道路距离，因此满足可采纳性条件，能够保证 A* 搜索结果的最优性。为了生成风格不同的候选路线，系统进一步使用加权启发策略，通过调整启发权重 ϵ 生成多条差异化路线供后续模型选择。

2.5.3 大语言模型辅助决策

大语言模型具备较强的自然语言理解、上下文整合与推理分析能力，适合用于对结构化算法结果进行再判断和解释。在本系统中，DeepSeek API 被用于多个辅助决策场景：在路径规划中结合候选路线和历史交通特征选择更合理的路线；在末端调度中结合聚类结果给出批次合并、拆分与优先级建议；在运力规划中对最小费用最大流结果进行人类可读分析。该方法并不替代传统优化算法，而是与确定性算法协同工作，提升决策结果的可解释性与实用性。

2.5.4 最小费用最大流算法（MCMF）

最小费用最大流算法用于求解在满足流量约束的前提下运输总成本最小的网络流分配问题。该算法常用于交通运输、供应链和资源调度场景中，能够同时处理容量约束与费用约束。在本系统中，全国物流干线网络被抽象为带容量与单位费用的有向图，仓库、枢纽站与中转站构成节点，线路构成边，订单需求则对应待分配的流量。通过 MCMF 求解，系统能够生成跨区域运输的全局运力分配方案，为管理员提供全国层面的资源配置支持。

2.6 本章小结

本章系统梳理了智能物流管理系统实现过程中涉及的关键技术。首先介绍了微服务治理、网关鉴权、服务调用与消息通信等基础技术；其次说明了 MyBatis-Plus、MySQL 与 Redis 在后端数据管理中的作用，以及 Vue3、Vite、Element Plus 和 Leaflet 在前端构建中的应用；随后对 Docker 与 Docker Compose 的容器化部署能力进行了说明；最后重点综述了 GraphHopper、A*、K-Means、MCMF 与大语言模型等智能算法相关技术。上述技术为后续系统设计与实现提供了完整的理论依据和工程基础。

c) 3 系统设计

本章主要介绍智能物流系统从需求设计、架构设计到前后端功能部件细化的全过程。在第一节，我们分析该系统涉及的角色用例、功能需求、非功能需求等。在第二节，我们介绍该系统的架构设计、技术栈选型等。在第三节和第四节，我们就前后端分别详细论述该系统的实现计划。最终，我们在第五节给出本部分的总结。

3.1 需求分析

3.1.1 总体分析

物流行业来自人类的工业运输与商业贸易需求，当今主要有两种表现形式：以合约委托的方式服务于企业；以计件付费的方式服务于个人。为了使我们的项目具备普遍意义，本系统分别考虑如下三种常见服务模式。

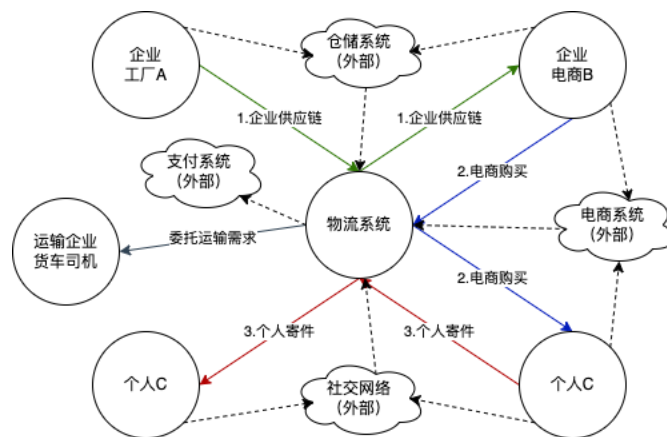


图 物流系统常见服务模式

如图所示，企业间通常会依据仓储系统的信息，将供应链的运输业务外包给物流系统。商品到达电商企业后，会通过淘宝、京东、拼多多等电商系统零售给个人。而个人也可能出于个人需求通过微信小程序等入口寄件，或针对闲鱼上的个人二手交易发起物流订单。物流公司通常会有分布在全国各地的仓库与服务网点，并将仓库/网点间的运输任务委托给运输企业或个体货车司机。

综上所述，物流系统的参与方主要为四方：作为客户的工厂、电商；作为客户的个人；作为平台的物流仓库、网点；作为第三方服务提供者的运输企业、货车司机。可能和本系统交互的外部系统包括：仓储系统、电商系统、社交网络、

支付系统。为了专注物流系统的本质，本项目略过支付系统，并为其他必要的外部接口仅提供最简单的实现。我们保留 API 接入功能和抽象层，只需通过 adaptor 模式进行简单的适配，即可接入这些外部系统。

以其中最复杂的电商购买模式为例，我们分析在此过程的流程及各个角色参与步骤。

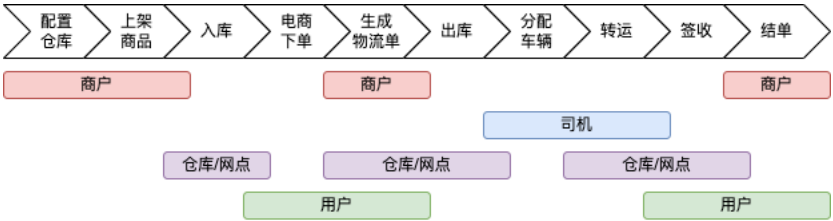


图 电商购买模式流程

如图所示，物流系统的单次服务以物流单为核心。在前期，商户在物流平台中选择自己所需位置的仓库，并配置各个商品的库存分布。物流公司旗下的仓库/网点提供入库、出库、转运、配送等全流程服务。用户在电商下单后，智能物流系统做出规划、生成物流单，并广播给商户、仓库和用户，并指派司机或下达委托。司机可以接受委托，并完成转运，在此过程中车辆定位将被物流系统全流程追踪。最终，终端网点完成配送，用户签收并向商户完成结单。

3.1.2 功能需求-个人用户用例

个人用户主要是基于地址的物品收发以及在电商中采购物品。

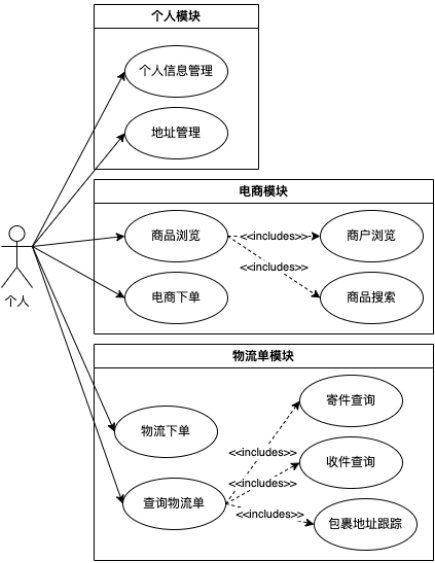


图 个人用户用例图

如图所示，个人用户需要地址管理、商品浏览、电商下单、物流下单、查看物流单等功能。

3.1.3 功能需求-商业用户用例

工厂/电商的主要运输对象是仓库中的批量商品。相比于个人，工厂/商户不使用地址，而是使用仓库作为地址。商户还可以在多个仓库间转运商品，以优化库存管理和发货成本。智能物流系统应该能够帮助商户自动规划库存分配。

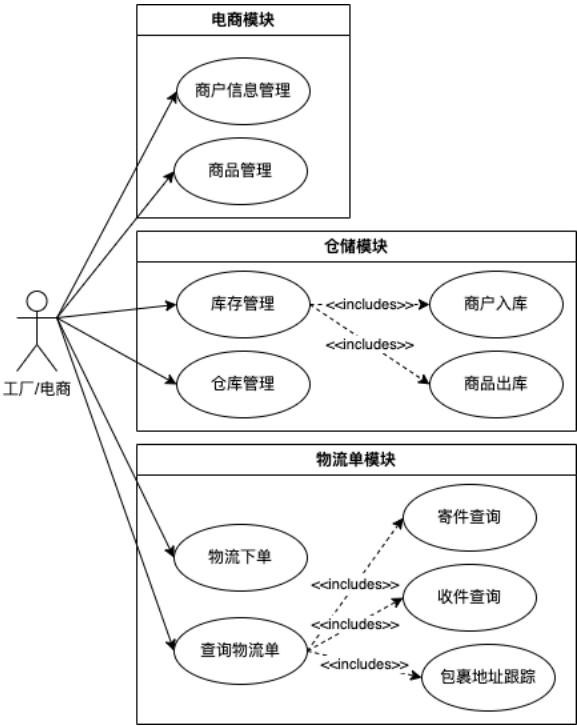


图 商业用户用例图

如图所示，商业用户需要管理商户信息、商品信息，还需要管理各个商品的库存状况。对于物流服务，商业用户与个人用户需求类似。

3.1.4 功能需求-司机用例

司机主要负责完成物流过程中的仓库到仓库的点到点运输，智能物流系统应该能基于路线、费用等问题自动规划出订单的最优安排，并为司机自动规划最优运输路线。由于司机车辆可能来自机构统一管理，因此需要额外设计车辆管理相关功能。

通过接入第三方 API，可以将外部平台的司机导入到本系统中，本系统同样可以为这些司机提供智能规划服务。

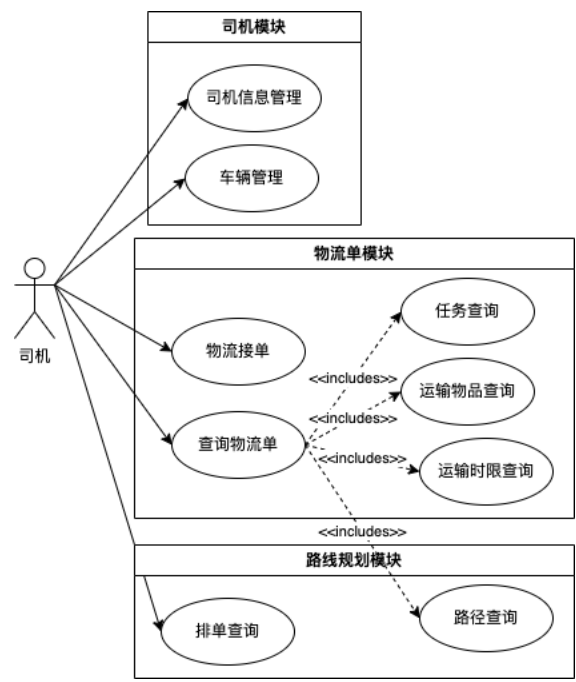


图 司机用户用例图

如图所示，司机用户需要管理司机身份信息、车辆信息，能够查询平台下发的订单，且可自行结单。此外，司机可以查询到系统智能规划的路线、当前路线上所载物品信息、当前任务起点终点、任务时限等信息。

3.1.5 功能需求-仓库/网点用例

仓库网点负责完成入库、出库、转运、派送等任务，这些任务可能通过物流公司配发的扫码设备采集包裹信息后自动触发至本系统中。

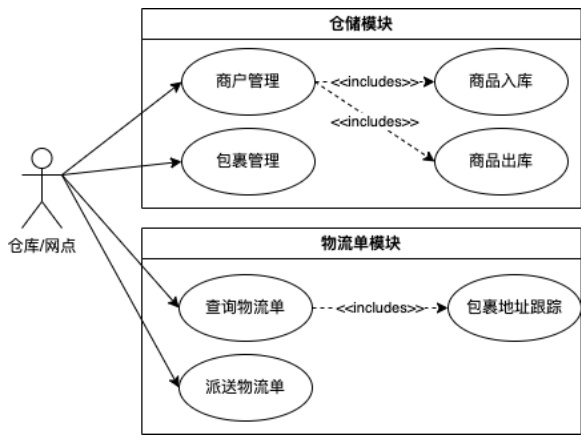


图 仓库/网点用例图

如图所示，仓库网点需要实现商户管理功能，为商家提供仓库托管、自动入库出库服务。此外，仓库网点需要实现包裹管理功能，并能查询物流单的进度。

最后，作为物流链条的末端，该角色需要派送物流单的功能，以实现签收结单。

3.1.6 非功能需求

本项目致力于通过微服务架构解决现实世界中物流系统数据量大、使用环境复杂、崩溃后果严重等问题，因此需要解决如下非功能需求：

- 1. 高并发：该系统可以通过横向扩展、弹性扩容等技术，合理应对高强度、随时变化的并发规模。
- 2. 高可用：该系统可以容忍部分节点的故障、自动负载均衡并将任务迁移到可用节点。
- 3. 可扩展性：该系统可以轻松接入外部系统及数据，可以轻松实现新的功能以应对日新月异的功能需求。
- 4. 用户友好：该系统在 UI 设计上简洁易懂、易于使用。
- 5. 安全性：该系统为用户提供完善的鉴权机制与权限控制，避免恶意用户的风险操作。

3.2 架构设计

本系统采用基于微服务架构的分层设计思想，将复杂物流业务拆分为多个职责清晰的独立服务，并通过统一网关、服务注册发现、同步调用与异步消息机制实现协同。整体架构可划分为接入层、业务层、通信层、存储层和编排层五个部分，其总体关系如图 3-1 所示。

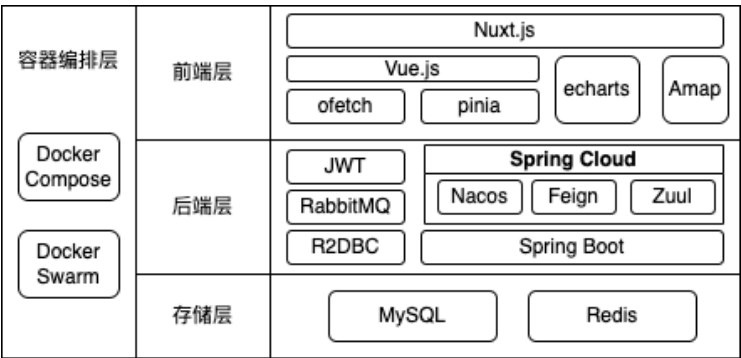


图 架构总览图

接入层由前端单页应用、Nginx 与 API 网关组成。前端采用 Vue3 构建多角色操作界面，并通过网关访问后端服务；网关服务作为系统唯一入口，负责请求路由、JWT 鉴权和用户身份透传。业务层由认证服务、用户服务、顾客服

与库存服务、订单服务、司机服务、物流服务及网关服务组成，各服务统一注册到 Nacos 中，实现服务发现与动态路由。通信层同时支持两类协作方式：一类是基于 OpenFeign 的同步服务调用，用于地址校验、库存查询、路线分配等即时业务协同；另一类是基于 RabbitMQ 的异步消息通信，用于库存扣减、配送创建、调度入池和状态联动等最终一致性场景。存储层采用 MySQL 作为持久化数据库，并使用 Redis 作为缓存与高频数据加速介质。编排层基于 Docker Compose 对数据库、中间件、后端微服务和前端容器进行统一部署，其中 logistics-service 还需挂载 OSM 地图数据和 GraphHopper 图缓存目录，以支持真实路网路径规划功能。

3.3 后端设计

基于前面的用例分析，我们将后端分为系统管理模块、个人模块、电商模块、司机模块、仓储模块、物流单模块、物流规划模块，共 7 个微服务。接下来，我们将依照这些微服务分别分析他们的数据库设计和 RESTful API 设计。

3.3.1 系统管理模块设计

本模块主要负责用户管理、登录鉴权等基本功能。主要设计用户实体类。对应的模型表如下所示：

表 3-1：User 模型属性表

模型名	字段名	数据类型	主要参数	描述或用途
User	id	CharField	max_length=128, primary_key=True, null=False	个人 ID
	username	TextField	max_length=100, null=False	用户名
	secret	TextField	max_length=100, null=False	密码哈希值
	permission	IntegerField	null=False	权限 Flag

同时我们可以给出对应的 API，如下所示：

表 3-2：系统管理模块 API 表

Method	URI	输入参数	输出参数	描述或用途
--------	-----	------	------	-------

POST	/signin	用户信息	注册
POST	/login	用户信息	登录
POST	/logout	用户信息	注销
GET	/user	用户信息	获取用户信息
PUT	/user/{id}	用户 ID	修改用户权限
DELETE	/user/{id}	用户 ID	删除用户

3.3.2 个人模块设计

本模块主要需要实现个人信息管理与地址管理的功能。其中涉及个人与地址两个实体类。个人与地址间是一对多的关系。因此，对应的模型表如下所示：

表 X：Person 模型属性表

模型名	字段名	数据类型	主要参数	描述或用途
Person	id	CharField	max_length=128, primary_key=True, null=False on_delete=models.CASCADE,	个人 ID
	user_id	ForeignKey	related_name='persons'	所属用户
	name	TextField	max_length=100, null=False	姓名
	phone	TextField	max_length=100, null=False	联系电话

表 X：Address 模型属性表

模型名	字段名	数据类型	主要参数	描述或用途
Address	Id	CharField	max_length=128, primary_key=True, null=False	地址 ID
	address	TextField	max_length=100, null=False on_delete=models.CASCADE,	地址文本
	person_id	ForeignKey	related_name='addresses'	所属个人

同时我们可以给出对应的 API，如下所示：

表 X：个人模块 API 表

Method	URI	输入参数	输出参数	描述或用途
GET	/person/{id}	个人 ID	个人信息	获取个人信息
POST	/person	个人信息		添加个人信息
PUT	/person	个人信息		修改个人信息
DELETE	/person/{id}	个人 ID		删除个人信息
GET	/person/{id}/address	个人 ID	个人地址	获取个人地址
POST	/person/address	个人地址		添加个人地址
PUT	/person/address	个人地址		修改个人地址
DELETE	/person/address/{id}	地址 ID		删除个人地址

3.3.3 电商模块设计

本模块主要需要实现商户信息管理、商品管理、商户浏览、商品搜索的功能。其中涉及商户、商品、仓库三个实体类。商品与仓库间、商户与仓库间是多对多的关系，因此实现为商品-仓库、商户-仓库关系表。而商户与商品间是一对多的关系，无需关系表。因此，对应的模型表如下所示：

表 X：Shop 模型属性表

模型名	字段名	数据类型	主要参数	描述或用途
Shop	id	CharField	max_length=128, primary_key=True, null=False	商户 ID
	user_id	ForeignKey	on_delete=models.CASCADE, related_name='addresses'	所属用户
	name	TextField	max_length=100, null=False	商户名称
	phone	TextField	max_length=100, null=False	联系电话

表 X：Goods 模型属性表

模型名	字段名	数据类型	主要参数	描述或用途
-----	-----	------	------	-------

Goods	Id	CharField	max_length=128, primary_key=True, null=False	地址 ID
	shop_id	ForeignKey	on_delete=models.CASCADE, related_name='goods'	所属个人

表 X: RepoShop 关系属性表

模型名	字段名	数据类型	主要参数	描述或用途
RepoShop	Id	CharField	max_length=128, primary_key=True, null=False	ID
	shop_id	ForeignKey	on_delete=models.CASCADE, related_name='goods'	商户 ID
	repo_id	ForeignKey	on_delete=models.CASCADE, related_name='goods'	仓库 ID
	capacity	IntegerField	null=False	容量

表 X: RepoGoods 关系属性表

模型名	字段名	数据类型	主要参数	描述或用途
RepoGoods	Id	CharField	max_length=128, primary_key=True, null=False	ID
	goods_id	ForeignKey	on_delete=models.CASCADE, related_name='goods'	商品 ID
	repo_id	ForeignKey	on_delete=models.CASCADE, related_name='goods'	仓库 ID

同时我们可以给出对应的 API，如下所示：

表 X: 商户模块 API 表

Method	URI	输入参数	输出参数	描述或用途
GET	/shop/{id}	商户 ID	商户信息	获取商户
POST	/shop	商户信息		添加商户

PUT	/shop	商户信息		修改商户
DELETE	/shop/{id}	商户 ID		删除商户
GET	/shop/{id}/repo	商户 ID	仓库容量	获取仓库列表
PUT	/shop/{id}/repo/{id}	商户 ID、仓库 ID		修改仓库容量
GET	/shop/{id}/goods	商户 ID	商品信息	获取商品列表
GET	/shop/search/{text}	搜索关键词	商品信息	搜索商品
GET	/shop/goods/{id}	商品 ID	商品信息	获取商品详情
POST	/shop/goods	商品信息		添加商品
PUT	/shop/goods	商品信息		修改商品
DELETE	/shop/goods/{id}	商品 ID		删除商品
GET	/shop/goods/{id}/repo	商品 ID	商品库存	获取商品库存
PUT	/shop/goods/{id}/repo/{id}	商品 ID、仓库 ID		修改商品库存

3.3.4 司机模块设计

本模块主要需要实现司机信息管理、车辆管理、排单查询等功能。司机信息管理的功能需要司机实体表。车辆管理的功能需要车辆实体表和司机-车辆关系表。排单查询的功能需要司机任务实体表。因此，对应的模型表如下所示：

表 X：Driver 模型属性表

模型名	字段名	数据类型	主要参数	描述或用途
Driver	id	CharField	max_length=128, primary_key=True, null=False	司机 ID
	user_id	ForeignKey	on_delete=models.CASCADE, related_name='drivers'	所属用户
	name	TextField	max_length=100, null=False	司机名称
	phone	TextField	max_length=100, null=False	联系电话

表 X：Vehicle 模型属性表

模型名	字段名	数据类型	主要参数	描述或用途
-----	-----	------	------	-------

Vehicle	id	CharField	max_length=128, primary_key=True, null=False	车辆 ID
	name	TextField	max_length=100, null=False	车牌号
	capacity	IntegerField	null=False	容量

表 X: DriverVehicle 关系属性表

模型名	字段名	数据类型	主要参数	描述或用途
DriverVehicle	id	CharField	max_length=128, primary_key=True, null=False	ID
	driver_id	ForeignKey	on_delete=models.CASCADE, related_name='addresses'	所属司机
	vehicle_id	ForeignKey	on_delete=models.CASCADE, related_name='addresses'	所属车辆

表 X: DriverTask 模型属性表

模型名	字段名	数据类型	主要参数	描述或用途
DriverTask	id	CharField	max_length=128, primary_key=True, null=False	任务 ID
	driverVehicle_id	ForeignKey	on_delete=models.CASCADE, related_name='addresses'	所属司机-车辆
	packet_id	ForeignKey	on_delete=models.CASCADE, related_name='addresses'	所属包裹
	src_addr	TextField	max_length=100, null=False	联系电话
	rest_addr	TextField	max_length=100, null=False	网点地址
	deadline	DateTime	null=True	时间限制

同时我们可以给出对应的 API，如下所示：

表 X: 司机模块 API 表

Method	URI	输入参数	输出参数	描述或用途
--------	-----	------	------	-------

GET	/driver/{id}	司机 ID	司机信息	获取司机
POST	/driver	司机信息		添加司机
PUT	/driver	司机信息		修改司机
DELETE	/driver/{id}	司机 ID		删除司机
GET	/driver/{id}/vehicle	司机 ID	车辆信息	获取司机的车辆
POST	/driver/{id}/vehicle/{id}	司机 ID、车辆 ID		司机添加车辆
DELETE	/driver/{id}/vehicle/{id}	司机 ID、车辆 ID		司机删除车辆
GET	/driver/vehicle/{id}	车辆 ID	车辆信息	获取车辆信息
POST	/driver/vehicle	车辆信息		添加车辆
PUT	/driver/vehicle	车辆信息		修改车辆
DELETE	/driver/vehicle/{id}	车辆 ID		删除车辆
GET	/driver/{id}/task	司机 ID	任务信息	查看排单

3.3.5 仓储模块设计

本模块主要需要实现仓库管理、入库出库、包裹管理的功能。对于仓库管理的功能，我们已在电商模块实现。对于另外两个功能，我们需要仓储实体类。如果我们观察包裹的瞬时状态，仓储和包裹是一对多的关系，因此不需要关系类。因此，对应的模型表如下所示：

表 X: Repo 模型属性表

模型名	字段名	数据类型	主要参数	描述或用途
Repo	id	CharField	max_length=128, primary_key=True, null=False	仓库 ID
	user_id	ForeignKey	on_delete=models.CASCADE, related_name='addresses'	所属用户
	name	TextField	max_length=100, null=False	网点名称
	phone	TextField	max_length=100, null=False	联系电话
	address	TextField	max_length=100, null=False	网点地址

同时我们可以给出对应的 API，如下所示：

表 X：仓库模块 API 表

Method	URI	输入参数	输出参数	描述或用途
GET	/repo/{id}	仓库 ID	仓库信息	获取仓库
POST	/repo	仓库信息		添加仓库
PUT	/repo	仓库信息		修改仓库
DELETE	/repo/{id}	仓库 ID		删除仓库
GET	/repo/{id}/shops	仓库 ID	商户信息	获取托管商户
GET	/repo/{id}/goods	仓库 ID	商品信息	获取托管商品
GET	/repo/{id}/packets	仓库 ID	包裹信息	获取库中包裹
POST	/repo/{id}/goods/{id}	仓库 ID、数量、 商品 ID		商品入库出库
POST	/shop/{id}/packets	仓库 ID、包裹 ID		包裹入库出库

3.3.6 物流单模块设计

物流单模块负责订单主记录、物流路线以及运输状态的承载，是连接订单履约与配送执行的重要数据基础。原有表格存在复制错误，未能反映真实实现中的字段设计。结合项目实际数据模型，本节对订单主表和物流路线表进行重新说明，并给出对应的核心接口设计。

表 X：order_info 实际字段表

字段名	数据类型	描述
id	BIGINT AUTO	PK 订单 ID
order_no	VARCHAR(32)	订单号（唯一）
customer_id	BIGINT	顾客 ID
shop_id	BIGINT	商户 ID
address_id	BIGINT	收货地址 ID
total_amount	DECIMAL(10,2)	订单总金额
order_status	TINYINT	0=已取消，1=待发货，2=待揽

		件，3=运输中，4=已送达
payment_status	TINYINT	0=未支付，1=已支付
warehouse_id	BIGINT	发货仓库 ID
origin_hub_id	BIGINT	起始中转站 ID
dest_hub_id	BIGINT	目标中转站 ID
flow_plan_id	BIGINT	关联的全国运力规划 ID
inter_city_batch_id	BIGINT	关联的干线批次 ID
payment_time / shipping_time / complete_time	DATETIME	支付、发货与完成时间
remark	VARCHAR(500)	备注

表 X：物流单模块核心接口表

Method	Path	描述
POST	/api/order	创建订单
GET	/api/order/{id}	获取订单详情
GET	/api/order/list	分页查询订单
PUT	/api/order/{id}/ship	商户发货
PUT	/api/order/{id}/status	更新订单状态
DELETE	/api/order/{id}	取消订单
GET	/api/logistics/routes/order/{orderId}	按订单查询路线详情
GET	/api/logistics/routes/{routeId}/journey	获取物流全程轨迹
PUT	/api/logistics/routes/{routeId}/status	更新路线状态

3.3.7 物流规划模块

物流规划模块是系统智能能力的核心承载者，主要负责路径规划、中转站分配、调度池管理、批次创建、末端调度预览以及全国网络运力规划等功能。结合项目实际实现，本节重新给出物流规划模块的核心数据与接口描述。

表 X：logistics_route 实际字段表

字段名	数据类型	描述
-----	------	----

id	BIGINT AUTO	PK 路线 ID
route_no	VARCHAR(32)	路线号（唯一）
order_id / delivery_id / driver_id	BIGINT	关联订单、配送单与司机
warehouse_id / batch_id / hub_id	BIGINT	关联仓库、批次与中转站
segment_type	TINYINT	0=普通路线，1=干线段，2=末端段
group_index / stop_count	INT	批次分组编号与停靠点数量
waypoints	TEXT	停靠点 JSON 数据
start_address / end_address	VARCHAR(255)	起终点地址
start_lat / start_lng / end_lat / end_lng	DOUBLE	起终点坐标
current_lat / current_lng	DOUBLE	司机当前位置
route_status	TINYINT	-1=待激活，0=待出发，1=运输中，2=已到达，3=异常
estimated_arrival_time actual_arrival_time	/ DATETIME	预计到达与实际到达时间
planned_route	LONGTEXT	GeoJSON 规划路线数据
receiver_name / receiver_phone	VARCHAR	收件人信息

表 X：物流规划模块核心接口表

Method	Path	描述
POST	/api/logistics/routes	创建物流路线并触发路径规划
POST	/api/logistics/routing/assign-hubs	分配起始与目标中转站
GET	/api/logistics/hubs	获取中转站列表
GET	/api/logistics/dispatch/pool	获取待调度订单池
GET	/api/logistics/dispatch/preview	K-Means + LLM 调度预览
POST	/api/logistics/dispatch/execute	执行调度并创建批次
POST	/api/logistics/dispatch/national/plan	触发全国运力规划
GET	/api/logistics/flow-plan/latest	获取最新运力规划详情

POST /api/logistics/track/location 上报 GPS 坐标

3.3.8 物流中转站模块设计

物流中转站模块用于管理城市级中转站（Hub）的基础信息，为 Hub-and-Spoke 配送模式提供节点数据支撑。中转站既承担订单集散与批次激活的职责，也是末端调度和干线到站联动的关键节点。

表 X：logistics_hub 模型属性表

字段名	数据类型	描述
id	BIGINT PK	中转站 ID
name	VARCHAR(100)	中转站名称
address	VARCHAR(255)	详细地址
latitude / longitude	DOUBLE	地理坐标
region	VARCHAR(50)	所属区域
max_capacity	INT	最大容量（件/日）
current_load	INT	当前负载量
status	TINYINT	0=正常，1=满载，2=关闭
remark	VARCHAR(255)	备注

表 X：中转站模块核心接口表

Method	URI	描述
GET	/api/logistics/hubs	获取中转站列表
GET	/api/logistics/hubs/nearest?lat=&lng=	查找最近中转站
GET	/api/logistics/hubs/{id}	获取中转站详情
POST	/api/logistics/hubs	新建中转站
PUT	/api/logistics/hubs/{id}	更新中转站信息
DELETE	/api/logistics/hubs/{id}	关闭中转站

3.3.9 全国干线网络模块设计

全国干线网络模块用于维护跨区域物流运输的枢纽层级结构和干线连接关系，为

全国运力规划算法提供基础拓扑输入。该模块通过全国枢纽表与连接表刻画不同层级 Hub 之间的运输能力与费率关系。

表 X: national_hub 模型属性表

字段名	数据类型	描述
id	BIGINT PK	全国枢纽 ID
hub_name	VARCHAR(100)	枢纽名称
hub_level	TINYINT	0=全国枢纽, 1=省级中心, 2=城市配送中心
city	VARCHAR(50)	所在城市
latitude / longitude	DOUBLE	地理坐标
daily_capacity	INT	日处理容量
current_waiting	INT	当前待发车件数
estimated_peak	INT	预计最大负载

表 X: hub_link 运输连接属性表

字段名	数据类型	描述
id	BIGINT PK	连接 ID
from_hub_id	BIGINT FK	起点枢纽
to_hub_id	BIGINT FK	终点枢纽
transport_mode	VARCHAR(20)	运输方式（公路/铁路/航空）
distance_km	DOUBLE	运输距离
capacity_daily	INT	日运输容量
base_cost_per_unit	DECIMAL	单位货物基础费率

3.3.10 配送批次与调度池模块设计

配送批次与调度池模块是 Hub-and-Spoke 执行链路中的核心运行单元，负责将待配送订单聚合为批次，并为末端配送和干线运输提供批量化组织能力。调度池用于维护待分配订单，批次表和批次项表则分别描述批次整体与订单级停靠安排。

表 X: logistics_batch 批次属性表

字段名	数据类型	描述
id	BIGINT PK	批次 ID
batch_no	VARCHAR(32)	批次编号（唯一）
warehouse_id	BIGINT	发货仓库 ID
hub_id	BIGINT	目标中转站 ID
trunk_route_id	BIGINT	干线路线 ID
batch_status	TINYINT	0=待发车，1=干线运输中，2=已到 Hub，3=末端配送中，4=已完成
total_orders	INT	批次订单数量
use_hub	TINYINT	是否经中转站
vrp_algorithm	VARCHAR(20)	批次算法标识
total_distance	DOUBLE	批次总距离

表 X: logistics_batch_item 批次项属性表

字段名	数据类型	描述
id	BIGINT PK	批次项 ID
batch_id	BIGINT FK	所属批次
order_id	BIGINT FK	订单 ID
route_id	BIGINT FK	对应末端路线 ID
visit_sequence	INT	大组内排序
stop_sequence	INT	停靠顺序
end_lat / end_lng	DOUBLE	收货坐标
end_address	VARCHAR(255)	收货地址
item_status	TINYINT	0=待配送，1=配送中，2=已送达

表 X: dispatch_pool 调度池属性表

字段名	数据类型	描述
id	BIGINT PK	池记录 ID
order_id	BIGINT	订单 ID

shop_id	BIGINT	商户 ID
warehouse_id	BIGINT	发货仓库 ID
end_address	VARCHAR(255)	收货地址
end_lat / end_lng	DOUBLE	收货坐标
status	TINYINT	0=待调度，1=已调度
batch_id	BIGINT	已调度的批次 ID

3.4 前端设计

本项目依照后端微服务的划分设计前端页面。

3.5 本章小结

本章从需求分析、总体架构、核心模块设计与前端设计四个方面对系统进行了系统性说明。首先，基于顾客、商户、司机与管理员等多角色业务需求，明确了系统的功能边界与非功能目标；其次，围绕微服务架构给出了由接入层、业务层、通信层、存储层和编排层组成的整体架构设计；随后，对用户认证、商品与仓储、订单与路线、中转站、全国干线网络、批次与调度池等关键数据模型和接口进行了模块化设计；最后，对前端多角色页面结构进行了简要规划。上述设计为后续系统实现章节提供了清晰的结构依据，也体现了本系统在 Hub-and-Spoke 物流网络与智能调度场景下的整体方案设计能力。

d) 4

系统实现

本章在前述系统设计的基础上，详细阐述基于微服务架构的智能物流管理系统的具体实现过程。内容将涵盖开发环境的搭建、后端各核心微服务模块的实现细节（特别是服务间通信、关键业务逻辑）、前端界面的构建、系统的集成部署以及测试验证。本章旨在展现如何将设计思想转化为实际可运行的系统，并突出关键技术的应用实践。

4.1 微服务环境搭建

为了保证各微服务在开发、测试与部署阶段具备一致的运行环境，系统首先完成了统一的微服务开发环境搭建。整体环境以 JDK 17、Maven 3.8+、Node.js 18+ 与 Docker 24+ 为基础，后端采用 Spring Boot 3.1.5、Spring Cloud 2022.0.4 与 Spring Cloud Alibaba 2022.0.0.0 构建微服务体系，前端采用 Vue3 + Vite 完成页面开发。

在服务治理方面，系统使用 Nacos 作为注册中心，各服务统一配置 `spring.cloud.nacos.discovery.server-addr=nacos:8848`，并注册至公共命名空间。服务启动后，网关可基于服务名完成动态发现与路由转发，从而避免手工维护静态地址。与此同时，网关在 `application.yml` 中配置了面向各业务服务的路径谓词，例如将 `/api/logistics/**` 路由至 `logistics-service`，将 `/api/shop/**`、`/api/stock/**`、`/api/warehouse/**` 等路径路由至 `shop-service`，从而形成统一入口。

系统服务端口采用固定划分方式：网关服务使用 8090 端口，认证服务、用户服务、顾客服务、商店与库存服务、订单服务、司机服务、物流服务分别使用 8081 至 8087 端口。这种命名与端口规划方式有助于在开发和容器化部署中快速识别服务角色，也便于后续日志排查与联调测试。中间件方面，系统同时引入 MySQL、Redis、RabbitMQ 与 Nacos 作为基础运行依赖，构成完整的开发运行环境。

【图 4-1 预留位置：微服务开发环境与服务端口分配示意图】

图 4-1 可展示系统中各微服务与中间件的部署关系、端口分配情况以及网关统一接入结构，用于说明运行环境的整体组织方式。

4.2 后端服务实现

4.2 认证与权限系统实现

在系统的业务层面，订单服务、商店与库存服务、司机配送服务共同构成了智能物流平台的核心执行链路。三者分别承担订单创建与编排、商品与库存资源管理、配送任务落地与状态推进等职责，并通过同步调用与异步消息协同完成跨服务业务闭环。本节从服务职责、关键流程与工程实现三个层面展开说明。

4.3.1 订单服务（order-service）

订单服务负责承接用户下单请求，并协调客户地址、商品状态、库存分布、配送创建与调度入池等多个环节，是系统业务编排最复杂的服务之一。在 `OrderServiceImpl.createOrder` 中，系统实现了面向电商物流场景的完整下单链路。首先，订单服务通过 Feign 调用 `customer-service` 校验收货地址的归属关系，以确保用户只能使用自身名下地址创建订单。随后，系统逐项遍历订单商品明细：一方面调用 `shop-service` 获取商品信息并校验商品状态，确保商品处于上架可售状态；另一方面继续调用库存查询接口，在多个仓库中寻找库存充足的发货仓。该阶段并不会立即扣减库存，而是将待扣减信息暂存在内存结构中，留待后续异步处理。完成全部订单项校验后，系统创建订单主记录，并将 `orderStatus` 设置为 1（待发货），将 `paymentStatus` 设置为 1（已支付）。由于本毕设主要聚焦物流流程与调度能力，故省略了真实支付网关接入，将下单视作支付已完成，从而简化了业务闭环实现。

订单记录落库后，系统继续批量写入订单项明细表，形成完整订单数据。随后，订单服务通过 RabbitMQ 发送库存扣减消息，由库存服务异步消费执行实际扣减，以实现最终一致性；若异步扣减失败，则消息进入死信队列，后续可由管理员或补偿任务介入处理。与此同时，订单服务还会发送“创建配送单”消息，由配送侧服务异步生成初始物流任务。为了支持后续中转调度，订单服务还会通过 Feign 调用 `logistics-service` 中的 `/routing/assign-hubs` 接口，为订单分配起始中转站与目标中转站，并更新订单的 `originHubId` 和 `destHubId` 字段。最后，系统再次发送 MQ 消息，将订单加入调度池，为后续批次规划和路线分配提供输入。

该流程体现了订单服务在微服务系统中的“业务编排中心”作用：它并不直接承担库存扣减、配送执行或路线计算等子任务，而是通过同步校验与异步消息相结合的方式，将多个服务串联成一个完整的业务事务。在设计层面，库存扣减采用MQ异步最终一致性而非同步强一致方式，主要考虑两点：其一，同步扣减会将下单接口性能直接绑定到库存服务响应时间，在高并发场景下容易形成系统瓶颈；其二，异步化后订单接口能够更快返回，提升用户体验。尽管这会引入极低概率的超卖风险，但系统通过死信队列与人工补偿机制将风险控制可接受范围内，因此更适合作为毕业设计中的工程折中方案。

4.3.2 商店与库存服务（shop-service）

商店与库存服务承担商品管理、仓库管理和库存维护等职责，是商户侧业务能力的核心承载者。服务不仅需要支撑商品上下架、仓库绑定、库存查询与修改等常规功能，还需要为订单服务提供可用于发货决策的实时资源信息。

在仓库管理方面，系统在 `WarehouseServiceImpl` 中集成了地理编码逻辑。当商户新增或修改仓库时，若提交了完整地址文本，系统会自动调用高德地图地理编码服务获取该地址的经纬度坐标，并写回仓库实体。其核心逻辑为：若 `fullAddress` 非空，则调用 `amapGeocodingService.geocode(fullAddress)` 解析坐标，并将结果写入 `latitude` 与 `longitude` 字段；若第三方地理编码调用失败，则仅记录警告日志，保留原有坐标信息而不中断主业务流程。该实现使仓库不仅是静态地址对象，也成为后续路径规划、距离计算和调度优化的地理节点。

在性能优化方面，服务对仓库列表查询采用了基于 `Spring Cache` 的缓存机制。系统通过 `@Cacheable(value = "warehouseAll")` 缓存仓库列表查询结果，以减少高频读请求对数据库的压力；对于新增、修改、删除等写操作，则使用 `@CacheEvict` 及时清除对应缓存，确保缓存与数据库状态保持一致。其底层缓存介质采用 `Redis`，既能保证读取性能，也具备较好的可扩展性。由于仓库信息在订单分仓、调度预览和路径规划中都会被频繁访问，因此该缓存策略对系统整体吞吐量提升具有明显作用。

在库存管理层面，服务维护商品与仓库之间的库存映射关系，并向订单服务提供“按商品查询库存分布”“按仓库返回可发货库存”等接口能力。订单服务通过调用这些接口即可完成最优发货仓的初步筛选，而无需感知库存表结构与更新细

节。由此可见，shop-service 不仅服务于商户前台的商品管理功能，也为后续订单履约与物流规划提供了基础数据支撑。

4.3.3 司机配送服务（driver-service）

司机配送服务负责承接配送任务生成、司机接单、过程状态推进以及末端履约落地等能力，是物流任务从“系统规划”走向“实际执行”的关键节点。服务围绕配送单状态机组织业务流程，使司机端页面、后台调度逻辑与订单履约状态之间能够保持一致。

系统设计的配送单状态机如下：待接单（0）→ 已接单（1）→ 取件中（2）→ 配送中（3）→ 已送达（4），同时允许在执行过程中进入已取消（-1）状态。通过这一有限状态机，系统能够清晰约束司机端的可执行操作，例如未接单任务不可开始取件，未完成取件则不可进入配送中状态，避免了前后端因状态不一致而导致的流程混乱。

在异步任务接入方面，DeliveryConsumer.java 是服务的重要消息消费入口。其一，当订单服务在商户发货后发送 QUEUE_DELIVERY_CREATE 消息时，司机服务会消费该消息并创建配送单，将任务放入“待接单大厅”，供合适司机认领；其二，当干线运输到达中转站后，物流服务会发送 QUEUE_LAST_MILE_ACTIVATE 消息，司机服务消费后创建末端配送单，支持最后一公里派送任务的激活。由此，服务既承担了首段配送单创建，也承担了中转后末端任务的接续，实现了对整条物流链路的闭环支撑。

在性能层面，服务针对司机首页高频访问的“进行中配送列表”进行了缓存优化。通过@Cacheable(value="inProgressDeliveries")，系统将司机当前进行中的配送任务缓存在 Redis 中，从而减少司机频繁刷新首页时对数据库的重复扫描，显著提升移动端页面加载速度。结合状态机约束与消息驱动的任务创建方式，driver-service 实现了从任务生成到实际配送执行的高效支撑。

4.2.1 认证与权限系统实现

系统认证与权限控制由认证服务与网关共同完成。其中，认证服务负责用户注册、登录与 JWT 令牌签发；网关负责统一鉴权、路径级权限校验以及用户身份透传。该设计将“身份建立”与“访问控制”进行解耦，一方面降低各下游服务的安全

实现复杂度，另一方面保证了微服务环境下权限控制的一致性。

在注册流程中，前端首先提交包含 `role` 字段的注册请求。后端在 `AuthServiceImpl.register()` 中先校验用户名唯一性，避免重复账号写入；随后使用 `BCrypt` 对明文密码进行加密，即调用 `passwordEncoder.encode()` 生成安全哈希值，再写入 `user` 表。系统以 `permission` 字段标识角色，其中普通客户映射为 2，商户映射为 3，司机映射为 4。当注册角色为商户时，系统会额外写入 `shop_info` 表，并将 `status` 初始化为 2，表示“待审核”；当注册角色为司机时，则额外写入 `driver_info` 表，同样设置为待审核状态。相比之下，普通客户注册后可直接使用系统功能，无需经过平台人工审核。由此可见，系统通过在注册阶段区分角色数据落库逻辑，实现了不同业务身份的准入控制。

在登录流程中，`AuthServiceImpl.login()` 首先根据用户名查询用户记录，并通过 `passwordEncoder.matches(password, user.getSecret())` 校验前端提交密码与数据库哈希值是否一致。若密码不匹配，则直接抛出业务异常，避免暴露更多账户信息。对于商户与司机这两类需要审核的角色，系统在密码校验通过后继续检查其审核状态：若状态为 2，则提示“待审核”，阻止其进入业务系统；若状态为 0，则提示“已禁用”，表示该账号已被平台冻结。只有状态合法的用户才会继续进入令牌签发阶段。认证成功后，系统调用 `jwtUtil.generateToken(user.getId(), user.getUsername(), roleCode)` 生成 JWT，其中封装了 `userId`、`username` 和 `roleCode` 等核心身份字段，作为后续微服务访问的统一认证凭据。

在网关侧，`JwtAuthFilter.java` 实现了统一的 JWT 鉴权流程。每次请求到达 Gateway 后，过滤器首先检查访问路径是否位于白名单中，例如 `/api/auth/login`、`/api/auth/register` 以及 Swagger 文档相关路径；若命中白名单，则直接放行。对于其余受保护请求，过滤器从 `Authorization` 头中的 `Bearer Token` 或自定义 `token` 头中提取令牌，并调用 `JwtUtil.validateToken()` 验证签名与有效期。在验证通过后，系统解析出 `userId`、`username` 与 `roleCode`，再根据请求路径执行 `hasPermission(path, roleCode)` 进行路径级权限判断。若权限匹配成功，网关会向下游服务追加 `X-User-Id`、`X-Username` 与 `X-Role-Code` 等请求头，下游微服务只需从请求头读取当前用户身份即可，无需再次解析 JWT，从而减少重复安全逻辑并提升了系统的可维护性。

系统的路径权限规则与角色职责一一对应：以/api/customer/为前缀的接口仅允许普通客户访问；/api/shop/与/api/stock/相关接口仅允许商户访问；/api/driver/仅允许司机访问；/api/logistics/ai/与/api/logistics/routes/同时开放给商户与司机，用于路线查询和智能辅助；而涉及全局运力调度的/api/logistics/dispatch/与/api/logistics/national-network/仅允许管理员访问。对于未显式配置的其他路径，系统默认在携带有效令牌的前提下放行。这种“白名单 + 令牌校验 + 路径授权”的组合机制，使系统既保证了接口安全性，又保留了良好的扩展能力。

总体而言，本系统的认证与权限实现遵循了“认证中心化、鉴权网关化、身份透传化”的设计思路。通过 BCrypt 保证密码存储安全，通过 JWT 支撑无状态认证，通过网关权限过滤实现统一访问控制，从而构建了适用于微服务架构的安全基础设施。

4.4 智能算法实现

4.5 前端实现

前端系统采用 Vue3 + Vite 构建多角色单页应用，并以 vue-router、Pinia、Element Plus 和 Leaflet 为核心技术支撑界面组织、状态管理与地图可视化。前端不仅负责展示顾客、商户、司机和管理员的业务页面，还承担登录态保持、角色访问控制、路线可视化与调度结果展示等任务。

在路由层面，系统通过全局路由守卫实现按角色的访问控制。页面跳转前，前端会从 userStore 中读取当前用户的 token 和 roleCode：若用户未登录且目标页面并非登录页或注册页，则强制跳转至登录页面；若访问页面声明了角色限制且与当前身份不匹配，则重定向回对应角色首页；若用户已登录但再次访问登录页，则直接跳转至其默认首页。系统将管理员、顾客、商户、司机分别映射到不同入口路径，从而形成清晰的多角色导航体系。上述实现可用伪代码描述为：判断是否存在令牌；若无令牌则限制访问；若存在令牌则校验角色是否匹配；匹配成功后放行，否则重定向至角色默认页面。

在地图与物流可视化方面，前端通过 RouteMap.vue 组件渲染后端返回的 GeoJSON 路线数据，并在地图上叠加起终点标记、实时司机位置以及路径折线；

LogisticsJourney.vue 则以时间线形式展示干线段与末端段的履约状态，形成“时间线 + 地图”的物流全程展示方式。在管理员侧，NationalNetworkMap.vue 基于 Leaflet 和自定义曲线绘制逻辑展示全国枢纽拓扑图，不同级别 Hub 使用不同颜色区分，干线批次按状态着色，鼠标悬停时可查看节点名称、层级与负载信息。

在业务页面组织方面，管理员工作台由多个专题页面构成，其中 HubOperations.vue 用于展示揽收、入库、分拣、出库和发车等中转站作业流程；NationalBusiness.vue 通过页签切换提供干线调度、全国网络和运力规划等能力；LastMileBusiness.vue 则聚焦城市末端调度、配送批次与中转站管理。通过这种基于角色和业务主题的页面组织方式，前端能够在保持操作逻辑清晰的同时承载复杂调度场景。

【图 4-2 预留位置：前端多角色路由与页面结构示意图】

图 4-2 建议展示登录页、角色首页映射以及管理员调度工作台的主要页面结构，用于说明前端信息架构设计。

智能算法模块是本系统区别于传统物流管理系统的重要特征。系统在路网级路径规划、候选路线智能决策、末端批次聚类调度以及全国干线运力规划等场景中引入图算法、聚类算法、最小费用最大流与大语言模型能力，使物流系统不仅能够“记录业务”，还能够“辅助决策”。本节重点说明这些算法模块的实现方式与工程集成路径。

4.4.1 GraphHopper 路网集成

为了在真实道路网络上执行路径搜索，系统在 logistics-service 中集成了 GraphHopper 作为底层路网引擎。该组件能够读取 OpenStreetMap 格式地图数据，构建可供汽车导航使用的图结构，从而为上层 A*搜索与候选路线生成提供道路拓扑基础。

在配置层面，系统于 application.yml 中定义了 graphhopper.osm-path、graphhopper.graph-cache-path 及 profiles 等关键参数。其中，osm-path 指向本地存放的 OSM PBF 地图文件，graph-cache-path 指向图缓存目录，而 profiles 中定义了名为 car 的车辆配置，使用 fastest 权重策略。对应的 Spring Bean 初始化逻辑中，系统通过 setOSMFile(osmPath)与 setGraphHopperLocation(cachePath)设置输

入 文 件 和 缓 存 目 录 ， 并 注 册 `new Profile("car").setVehicle("car").setWeighting("fastest")` ， 最 终 调 用 `hopper.importOrLoad()`完成路网导入或缓存加载。

需要说明的是，系统首次启动时需要 OSM 数据进行建图，该过程通常需要数分钟；而在后续重启时，若缓存目录中已有构建好的图索引，则可直接加载缓存，显著缩短启动时间。通过这一集成方式，系统获得了接近真实地图环境的道路通行能力，为后续的路径算法与调度优化提供了可靠的数据底座。

4.4.2 A* 路径规划实现

在底层路网图之上，系统实现了基于 A*算法的路径规划策略 `AStarRouteStrategy`。该策略以启发式搜索代替纯粹的最短路遍历，在保证最优性的前提下提升了搜索效率，适用于城市道路网络中从起点到终点的快速路线求解。

算法实现中主要维护四类核心数据结构：其一，开放集合 `openSet` 采用优先队列实现，存储待扩展节点及其 `f` 值，并按 `f` 从小到大排序；其二，`gScore` 记录起点到当前节点的实际代价；其三，`cameFrom` 记录路径回溯关系，用于在搜索结束后重建完整路径；其四，`closedSet` 用于标记已处理节点，避免重复扩展。对于启发函数，系统采用 Haversine 球面距离估计当前位置到终点的直线距离。该启发函数具有可采纳性，不会高估真实道路距离，因此能够保证 A*搜索仍然得到最优解。

在标准 A*基础上，系统进一步实现了 `Weighted A*`机制，即在启发项前引入权重系数 `epsilon`。当 `epsilon > 1.0` 时，启发值被放大，算法会更激进地趋向终点，从而探索出与标准最优路径不同的候选路线。具体而言，系统通过设置 `epsilon=1.0`、`1.8` 与 `3.5` 生成多条具有风格差异的候选路径，为后续 LLM 智能决策提供备选空间。搜索完成后，系统从终点沿 `cameFrom` 链路逆向回溯到起点，并将节点序列转换为 `GeoJSON LineString` 格式返回，方便前端地图组件直接绘制。

此外，系统还对路线耗时进行了快速估算：在未引入历史交通覆盖前，以城市平均车速 `40km/h` 为基准，通过“总距离 ÷ 平均速度”计算出 `durationMs`。这一基准时长既可作为基础返回值，也可在后续被 LLM 依据历史拥堵经验进行修正。由此，`AStarRouteStrategy` 承担了系统路线规划的基础求解能力。

4.4.3 LLM 路径决策实现（LlmJudgeRouteStrategy）

为了克服传统最短路算法仅关注距离、难以融合经验性交通因素的局限，系统进一步实现了 LlmJudgeRouteStrategy，利用大语言模型在多候选路线之间进行综合判断。其核心思想并非让 LLM 直接“生成路线”，而是让图算法先生成若干候选解，再由 LLM 结合历史统计信息做出更贴近现实的决策。

具体流程如下：第一步，系统使用不同 `epsilon` 参数（1.0/1.8/3.5）调用加权 A* 生成三条候选路线，并按距离 1% 阈值去重，去除差异过小的重复路径；第二步，系统通过 RouteHistoryService 查询与当前时段相关的历史数据，包括当前小时平均速度、全天平均速度、延误统计、近 7 天异常路线次数以及历史慢速热点区域等；第三步，系统在 buildUserPrompt 中将候选路线摘要（距离、基准时间、节点数）、历史交通特征与输出格式约束组织为提示词，要求模型严格输出 JSON 结果；第四步，系统调用 DeepSeek API 并解析返回结果，得到诸如 selectedCandidate、reasoning、adjustedDurationMs、summary、warnings 与 confidenceLevel 等字段；第五步，系统以模型选中的候选路线作为最终结果，并使用 adjustedDurationMs 覆盖基础时间估计。

例如，当模型判断“候选路线 2 历史延误较少”时，可返回“全程约 12km，早高峰预计需 30 分钟”等可读摘要，并提示“途经区域历史拥堵”等风险信息。这种输出一方面可以被后端结构化消费，另一方面也能够直接转化为前端展示文案。若调用大模型失败、结果解析异常或返回结构不合法，系统会执行降级策略，直接返回 `epsilon=1.0` 对应的标准 A* 最优路线，从而保证接口始终具备稳定输出。由此，系统将确定性图搜索与概率性智能分析进行了有效结合，实现了“算法生成候选，模型进行裁决”的工程化路径决策模式。

4.4.4 K-Means 聚类调度预览

在末端配送场景中，系统提供了基于 K-Means 的调度预览能力，对待调度订单进行空间聚类，从而辅助管理员设计批次与司机分配方案。其预览接口为 GET /api/logistics/dispatch/preview，支持传入聚类数量 k、仓库 warehouseId、订单集合、是否跳过 LLM 分析等参数。

执行过程中，系统首先从调度池（dispatch_pool）中获取目标中转站下待调度的订单集合，并提取每个订单的收货坐标作为聚类输入。随后，系统执行 K-Means

算法，将空间上相近的订单归并到同一簇中，以降低单批次路线的绕行成本。聚类完成后，系统对每个簇内部进一步执行以仓库为起点的近似 TSP 排序，生成合理的派送先后顺序。若请求参数中 `skipLlm=false`，系统还会调用大语言模型对聚类结果进行二次分析，例如判断是否存在紧急订单应优先处理、是否建议合并或拆分某些批次，以及是否存在潜在超载或时间冲突风险。最终，接口返回 `DispatchPreviewDTO`，其中包含各聚类的订单列表、建议司机数量与 LLM 建议文本。

当管理员确认预览方案后，可进一步调用执行接口提交正式调度结果。系统会为每个聚类创建一个 `LogisticsBatch`，并生成对应的分段路线，其中末端路线以 `segment_type=2` 标识，初始状态设为 -1（待激活）。这一实现将空间聚类、启发式路径排序与智能文本建议结合起来，使管理员可以在执行前先获得可解释、可调整的调度方案。

4.4.5 MCMF 全国运力规划实现

在全国范围的干线运输规划中，系统引入最小费用最大流（Min-Cost Max-Flow, MCMF）算法，对跨仓、跨中转站的运力分配进行全局优化。管理员可通过 `POST /api/logistics/dispatch/national/plan?planDate=...` 触发某一日期的运力规划任务。

在网络建模上，系统首先引入超级源点 `S` 与超级汇点 `T`。从超级源点到各发货仓库的边表示日发货需求，容量设置为仓库日发货量；从发货仓库到其关联的起始中转站（`originHub`）的边表示本地集散能力，容量受仓库能力约束，边费用表示本地转运费；`Hub` 与 `Hub` 之间的边表示干线连接，其 `capacity_daily` 反映干线日运输能力，`edge_cost` 反映单位运输成本；而从目标中转站（`destHub`）到超级汇点的边则表示终端处理能力，容量由中转站日处理量决定。通过这一方式，系统将全国物流网络抽象为带容量与费用约束的有向流网络。

在正式运行 MCMF 之前，系统还引入了 LLM 辅助费率校准机制。系统会读取各干线边的历史运输数据，并调用大模型分析是否存在显著偏离同类线路的异常费率，例如某条线路单位运费异常偏高。模型建议会被封装为 `EdgeCostCalibrationDTO`，并用于修正干线边费用输入，从而使规划结果更贴近历史实际与管理经验。运算完成后，系统将各线路分配流量、总费用以及 LLM 分析报告写入 `flow_plan` 与 `flow_plan_item` 表中，供后续查询与展示。

在前端展示层，FlowPlan.vue 会显示规划日期、参考需求量、预估总费用等摘要信息，并通过折叠区域展示模型生成的人类可读分析，如瓶颈节点、费率异常与优化建议。同时，界面还会展示各干线路线的流量、容量与负载率，帮助管理员从宏观层面理解全国物流网络的资源配置情况。由此，系统实现了从数学优化模型到业务可视化决策的闭环。

4.4.6 Hub-and-Spoke 完整配送流程

基于上述算法与服务能力，系统最终形成了完整的 Hub-and-Spoke 配送闭环。首先，当商户下单后，订单服务会调用 /routing/assign-hubs 为订单分配起始与目标中转站，随后将订单写入调度池，标记为待调度。接着，管理员在末端调度界面调用 previewDispatch 预览方案，经确认后执行 executeDispatch。系统据此创建 LogisticsBatch，并同步生成干线路线（segment_type=1）与末端路线（segment_type=2），其中末端路线初始状态为-1，表示待激活。

在实际运输阶段，司机首先接单并执行干线运输，过程中持续上报 GPS 位置。当司机抵达中转站后，配送服务在 DeliveryService.arriveAtHub() 中触发到站逻辑：一方面发送“#13 HubArrival”消息，由物流服务消费后将对应末端路线状态从-1 更新为 0；另一方面继续发送“#14 LastMileActivate”消息，由司机服务消费并创建末端配送单，将其投入待接单大厅。随后，末端司机接单并按照 waypoints 多停靠点路线逐单配送，在每次送达后更新 BatchItem.itemStatus。当所有停靠点均完成后，若批次内全部 BatchItem 状态均为 2，系统最终将整批 Batch 更新为已完成（4）。

通过这一流程，系统实现了从下单、分配中转站、聚类调度、干线运输、到站激活、末端派送直至批次完成的完整履约链路，体现了“中心枢纽 + 辐射末端”物流模式在微服务架构下的工程化落地能力。

4.6 容器化部署实现

系统最终采用 Docker Compose 实现容器化集成部署，将数据库、中间件、后端微服务与前端静态资源统一纳入同一编排文件管理。整体依赖关系为：首先启动 MySQL、Redis、RabbitMQ 与 Nacos 等基础中间件，待其健康检查通过后再启动认证、用户、顾客、商店、订单、司机与物流等业务服务，随后启动网关服务，

最后由 Nginx 承载的前端容器对外提供页面访问入口。该顺序能够有效避免因依赖未就绪而导致的服务启动失败。

在服务镜像构建方面，各微服务目录均提供独立的 Dockerfile，通过多阶段构建或标准 JAR 打包方式生成运行镜像。物流服务由于需要进行真实路网计算，因此在容器编排中还额外挂载了本地地图数据卷与图缓存目录，分别用于保存 OSM 原始地图文件与 GraphHopper 路网缓存文件。与此同时，环境变量中还可配置路线策略类型，例如设置 ROUTING_STRATEGY=A_STAR 或 LLM_JUDGE，并通过环境变量注入 DEEPSEEK_API_KEY，从而满足不同运行策略下的部署需求。

前端部署采用 Nginx + Vite 构建产物方式完成。构建后的静态文件被复制到 Nginx 容器目录中，通过 try_files \$uri \$uri/ /index.html 支持单页应用路由回退；同时将 /api/ 请求统一反向代理至 gateway-service:8090，从而实现前后端分离部署与统一入口访问。该方案既保留了前端应用部署的简单性，也方便后续与网关服务协同运行。

【图 4-3 预留位置：Docker Compose 容器依赖关系图】

图 4-3 建议展示中间件、后端微服务、网关与前端容器之间的启动依赖关系，以及物流服务地图数据卷挂载方式，用于说明系统的整体部署结构。

本章围绕基于微服务架构的智能物流管理系统实现过程，系统性说明了开发环境搭建、认证与权限控制、核心业务服务、智能算法、前端界面以及容器化部署等关键内容。首先，系统完成了以 Nacos、Gateway、RabbitMQ、Redis 和 MySQL 为基础的微服务环境搭建；其次，通过认证服务与网关实现了基于 BCrypt 和 JWT 的统一认证鉴权机制；随后，围绕订单服务、商店与库存服务、司机配送服务展示了跨服务编排、异步消息驱动与缓存优化的业务实现路径；在算法层面，结合 GraphHopper、A*、K-Means、MCMF 与大语言模型能力，说明了系统如何在路径决策、批次调度与全国运力规划中提供智能化支持；最后，阐述了前端多角色页面实现方式以及基于 Docker Compose 的容器化部署方案。上述实现验证了第三章设计方案的可行性，也体现了本系统在复杂物流业务场景下的工程集成能力与智能优化特征。

e) 5

系统展示

5.1 并发性能测试

为了从系统运行效果层面验证前文设计与实现的有效性，本章从性能测试和多角色功能展示两个角度对系统进行综合说明。其中，性能测试部分重点考察高并发访问、缓存优化和异步消息机制对系统吞吐能力与响应效率的支撑作用；功能展示部分则围绕顾客、商户、司机和管理员四类核心用户，展示系统在真实业务链路中的交互界面与操作流程。通过本章内容，可以进一步体现该智能物流管理系统在工程实现上的完整性、可用性与可视化程度。

在测试方案设计上，系统选取了能够代表核心业务负载特征的四类接口进行验证。首先，针对 `GET /api/order/list` 接口实施并发访问，用于观察订单查询场景下的吞吐能力与响应稳定性；其次，对 `GET /api/warehouse/list` 分别在启用与关闭 Redis 缓存的条件下进行对比，以分析缓存机制对热点查询性能的改善效果；再次，对 `POST /api/logistics/routes` 路径规划接口进行压力测试，重点统计平均响应时间与 P99 指标，从而评估系统在实时路线计算场景中的处理能力；最后，在 `POST /api/order` 高并发下单场景中，结合消息队列消费情况验证系统是否能够通过异步削峰保障核心链路的稳定运行和最终一致性。

【图 5-1 预留位置：仓库列表接口缓存前后响应时间对比图】

图 5-1 可展示启用缓存后仓库列表接口平均响应时间显著降低的测试结果，以说明 Redis 缓存对热点查询的性能提升效果。

【图 5-2 预留位置：路径规划接口压力测试统计图】

图 5-2 建议展示路径规划接口在 A_STAR 策略下的响应时间分布与 P99 统计值，用于说明系统具备实时规划能力。

5.2 个人用户功能展示

个人用户侧主要面向普通顾客的购物与物流使用场景，覆盖注册登录、商品浏览、购物车下单、订单查询以及物流轨迹查看等核心功能。界面设计强调流程简洁、操作直观和信息反馈清晰，使用户能够在较低学习成本下完成从浏览商品到收货追踪的完整闭环操作。从论文展示角度看，该部分能够较好体现系统在电商消费

场景中的可用性和业务完整性。

在实际操作过程中，用户首先通过统一入口完成身份注册与登录，随后进入商品浏览页面完成搜索、筛选与详情查看，并在确认订单后创建购买请求。订单生成后，系统会根据仓库库存与物流规划结果自动进入履约流程，用户可在订单详情页实时查看当前状态、物流时间线以及地图轨迹信息。上述页面组合完整反映了顾客侧业务的主要交互路径。

【图 5-3 预留位置：用户注册与登录页面】

图 5-3 建议展示用户选择角色、填写注册信息以及登录系统的页面，用于说明系统多角色统一入口的设计方式。

【图 5-4 预留位置：商品浏览与下单确认页面】

图 5-4 可展示商品列表、筛选搜索、商品详情及下单确认信息，以说明顾客从浏览商品到创建订单的业务流程。

【图 5-5 预留位置：订单详情与物流轨迹页面】

图 5-5 建议展示订单状态标签、物流时间线和地图轨迹组件，用于体现系统对订单全过程可视化追踪的支持。

5.3 商户用户功能展示

商户用户侧主要承担商品管理、仓库管理、库存维护以及订单发货等业务，是连接电商交易与物流履约的重要环节。系统通过列表管理、弹窗编辑和批量操作等交互方式，支持商户完成商品信息维护、仓库绑定、库存配置及发货处理等核心操作。该部分展示不仅体现了商户端业务流程的完整性，也反映出系统在多仓管理和履约协同方面的工程实现能力。

在商户侧典型流程中，用户首先维护店铺商品信息并配置上下架状态，随后为商品绑定可发货仓库并设置各仓库存数量。当顾客下单后，商户可在待发货订单页中查看订单详情、执行发货确认，并由系统自动触发后续配送创建与物流流转。通过这一流程，商户端与订单服务、库存服务及物流服务形成紧密协同关系。

【图 5-6 预留位置：商户商品管理页面】

图 5-6 可展示商品列表、新增或编辑商品弹窗，用于说明商户如何维护商品信息及上下架状态。

【图 5-7 预留位置：仓库与库存管理页面】

图 5-7 建议展示仓库列表、仓库地址配置以及商品-仓库库存表格，以体现系统对多仓库库存管理的支持。

【图 5-8 预留位置：待发货订单管理页面】

图 5-8 可展示商户查看订单、执行发货以及触发配送创建的操作入口，用于说明商户端在履约流程中的关键作用。

5.4 司机用户功能展示

司机用户侧主要用于接收配送任务、执行运输导航、上报配送进度以及完成到站与送达确认，是物流计划落地执行的直接承载端。系统通过待接单列表、路线地图、配送详情与状态更新界面，分别支撑干线运输与末端配送两类业务任务。该部分能够直观展示智能物流系统如何将后台调度结果转化为司机可执行的运输流程。

具体而言，司机可在待接单大厅中查看系统生成的普通配送单或末端配送单，并根据任务信息完成接单操作。接单后，系统会根据后端规划结果在地图页面中展示起终点、推荐路线和实时位置，辅助司机执行导航；当车辆抵达中转站或完成末端配送后，司机可通过按钮操作推进配送状态，从而与订单履约状态形成同步更新。

【图 5-9 预留位置：司机待接单大厅】

图 5-9 建议展示普通配送单与末端配送单的待接单列表，用于说明司机如何认领系统分配的配送任务。

【图 5-10 预留位置：导航与路线地图页面】

图 5-10 可展示 Leaflet 地图、规划路线折线、实时司机位置与起终点标记，用于体现司机端地图导航能力。

【图 5-11 预留位置：中转站到达确认与多停靠点送达页面】

图 5-11 建议展示司机抵达中转站后的到达确认按钮以及末端多停靠点逐单完成界面，用于说明配送状态推进机制。

5.5 管理员功能展示

管理员侧是本系统全局管理、智能调度与网络规划能力的集中体现，主要涵盖基础数据维护、商户与司机审核、中转站管理、末端调度、干线批次控制以及全国运力规划等功能模块。相较于顾客、商户和司机侧，管理员侧更能体现系统的业务复杂度与创新性，是论文展示中最能够反映项目特色与技术价值的部分。

在管理员的典型操作路径中，系统首先提供整体运行概览与待审核用户管理能力，用于完成平台准入控制；随后，管理员可在中转站管理与作业台页面中查看各节点负载情况并控制批次流转；在末端调度页面中，系统结合聚类与大语言模型建议生成配送批次预览；在全国网络与运力规划页面中，则进一步展示跨区域干线资源配置结果。通过这些界面，系统完整体现了从局部节点管理到全国网络优化的层次化调度能力。

【图 5-12 预留位置：管理员仪表盘与用户审核页面】

图 5-12 可展示订单统计卡片、待审核商户与司机列表等基础管理界面，用于说明管理员对系统全局状态的把握能力。

【图 5-13 预留位置：中转站列表与 Hub 作业台页面】

图 5-13 建议展示中转站状态、当前负载、流水线作业步骤等内容，用于说明管理员如何管理中转站日常作业。

【图 5-14 预留位置：末端调度预览与配送批次详情页面】

图 5-14 可展示 K-Means 聚类结果、LLM 建议面板、批次详情时间线与地图，用于体现城市末端调度功能。

【图 5-15 预留位置：全国网络拓扑与运力规划结果页面】

图 5-15 建议展示全国 Hub 网络拓扑图、干线批次状态以及 MCMF 运力规划结果和模型分析文本，用于体现系统在全国物流网络层面的智能调度能力。

结论

本研究旨在应对传统物流管理系统在效率、成本和灵活性方面存在的挑战，通过引入先进的微服务架构和相关技术，设计并实现了一个现代化的智能物流管理系统。经过需求分析、系统设计、技术选型、编码实现以及测试验证等阶段，本文成功构建了一个功能原型系统，验证了所提出技术方案的可行性与有效性。

6.1 总结

本文围绕基于微服务架构的智能物流管理系统展开研究，针对传统物流平台在系统耦合度高、业务扩展困难、路径决策能力不足以及跨环节协同效率较低等问题，完成了从需求分析、总体设计到系统实现与展示验证的完整工作。研究过程中，本文首先梳理了顾客、商户、司机与管理员等多角色的业务需求，明确了系统在订单履约、仓储库存、配送执行、路径规划与调度优化等方面的核心目标，并在此基础上构建了面向复杂物流场景的系统方案。

在系统设计与实现过程中，本文采用微服务架构将复杂物流业务拆分为认证、用户、顾客、商店与库存、订单、司机、物流规划和网关等多个服务单元，并基于 Spring Boot、Spring Cloud 与 Spring Cloud Alibaba 完成服务治理。数据访问层采用 MyBatis-Plus 与 MySQL，缓存层采用 Redis，并通过 RabbitMQ 实现库存扣减、配送创建与调度入池等异步协作逻辑。前端采用 Vue3 + Vite、Element Plus、Pinia 与 Leaflet 构建多角色交互界面，最终通过 Docker Compose 实现系统容器化部署。

在关键功能实现方面，本文重点完成了基于 BCrypt 与 JWT 的统一认证授权机制、基于 OpenFeign 和 RabbitMQ 的服务间同步与异步协同机制，以及围绕订单履约流程的核心业务编排逻辑。在智能化能力方面，系统集成了基于 GraphHopper 真实路网的 A* 路径规划算法，引入大语言模型对多候选路线进行辅助裁决，并通过 K-Means 支撑末端调度预览，通过 MCMF 支撑全国运力优化。与此同时，系统构建了基于 Hub-and-Spoke 三级物流网络的完整履约流程，实现订单从下单、分配中转站、干线运输到末端配送的闭环执行。

从系统验证结果来看，本文构建的原型系统已经能够支撑顾客下单、商户发货、司机配送、中转站流转以及管理员全局调度等关键业务流程，并在多角色界面展示、缓存性能优化和路径规划响应能力等方面体现出较好的工程可行性。尤其是在真实路网路径规划、聚类调度预览和全国运力规划等功能上，系统展示出相较传统信息管理平台更强的智能辅助能力。

综上所述，本文完成了一个面向智能物流场景的微服务系统原型构建，并在工程实现中验证了微服务解耦、异步消息协同、缓存优化、真实路网规划与智能算法融合的可行性。该系统不仅能够满足顾客、商户、司机和管理员多角色的基本业务需求，也为微服务架构在智能物流领域的落地应用提供了具有实践价值的参考方案。

6.2 不足与展望

尽管本文已完成一个具备较完整业务链路和智能调度能力的系统原型，但受时间、数据来源与工程条件限制，当前系统仍存在一定不足，且在后续研究与实践中仍有较大的扩展空间。

（1）当前系统的主要不足体现在以下几个方面：

1. 功能覆盖与深度有限：当前系统虽已完成智能物流管理的核心链路，但仍存在进一步完善空间。例如，K-Means 聚类中的 k 值仍需要管理员在调度界面手工指定，缺乏基于历史数据的自适应推荐机制；大语言模型调用在高并发场景下存在响应延迟，当前仅通过回退至标准 A^* 路线作为降级方案，后续仍可引入本地轻量模型或规则引擎提高实时性；部分运力规划参数仍采用静态配置，尚未完全与真实运营数据联动。
2. 生产环境适应性：当前系统采用 Docker Compose 单机方式部署，虽然便于开发与演示，但在大规模生产环境下仍难以实现真正的弹性扩缩容与自动故障恢复。尤其是物流服务在加载大规模 GraphHopper 路网数据时会消耗较多内存资源，若扩展为多副本部署将带来更高的资源成本。此外，跨服务数据一致性目前主要依赖 RabbitMQ 与死信队列，在网络分区或极端异常场景下仍缺乏更完善的事务补偿机制，后续可考虑引入更系统化的 Saga 模式。
3. 数据来源与场景真实性仍有不足：系统虽然已经接入真实地图数据并模拟了

完整物流流程，但部分输入数据仍带有演示性质。例如，实时路况、司机位置上报、订单密度分布以及干线运输需求等数据仍主要来源于预设数据或简化模拟，尚未与真实运营平台实现持续联动。因此，系统在更复杂、更动态的真实业务环境中的鲁棒性与泛化能力仍有待进一步验证。

4. 测试维度仍需进一步完善：目前系统已完成核心功能验证与部分性能测试，但在安全性测试、异常链路测试、极端并发下的稳定性测试以及长时间运行场景下的资源占用分析等方面仍不够充分。后续若要推动系统向更高质量的软件工程成果靠近，还需建立更加系统化、自动化的测试体系。

（2）针对上述问题，未来可从以下几个方向继续开展优化与拓展：

针对上述不足，未来可以从以下几个方向进行深入研究和系统优化：

1. 进一步完善业务功能与智能辅助能力。后续可在现有订单、库存与配送流程基础上，引入更加精细化的库存预测、动态补货、异常预警和时效预测功能，并进一步优化大语言模型在路线建议、调度解释和风险提示中的作用，使系统在“可执行”基础上进一步具备“可预测、可解释、可优化”的能力。
2. 推进部署架构向生产级形态演进。当前系统以 `Docker Compose` 为主完成单机容器化编排，后续可逐步迁移至 `Kubernetes` 等平台，以获得自动扩缩容、故障自愈和滚动更新等能力。同时，还可结合服务监控、链路追踪与集中日志平台，进一步提升系统在生产环境下的可运维性。
3. 深化算法模型研究。未来可尝试将启发式路径规划、聚类调度与网络流优化进一步结合历史业务数据进行训练与校准，并探索引入更复杂的车辆路径问题（VRP）模型、时间窗约束模型和强化学习策略，以更贴近真实物流业务中的多目标优化需求。
4. 加强与外部系统和真实数据源的集成。后续可进一步对接真实电商平台接口、仓储设备数据、定位终端与地图服务，实现订单来源、轨迹上报和库存变化的实时化接入，从而提升系统展示结果与实际物流运营之间的一致性。
5. 持续提升系统智能化与自动化水平。除路线与调度优化外，系统未来还可以在智能客服、异常识别、运力预警、资源推荐和业务复盘分析等场景中进一步引入人工智能技术，从而推动物流系统由业务数字化向决策智能化演进。
6. 完善安全治理与测试体系。未来可进一步补充接口鉴权审计、敏感数据保护、

权限细粒度控制与自动化回归测试机制，并结合压力测试、故障注入测试和混沌工程思想，全面提升系统在复杂运行条件下的可靠性与安全性。

通过持续的迭代优化和技术探索，有望将本系统发展成为一个功能更强大、性能更优越、更贴近实际业务需求的智能化物流管理平台，为物流行业的数字化转型贡献力量。

参考文献

- [1] 中华人民共和国国家发展和改革委员会. 《“十四五”现代物流发展规划》[R]. 2022.
- [2] WANG L, ZHANG Y, GAO P. Research on Intelligent Logistics System Based on IoT Technology[C]//Journal of Physics: Conference Series. IOP Publishing, 2021, 1881(4): 042047.
- [3] LI J, WANG F. Big Data Analytics for Intelligent Logistics Systems: A Review[J]. IEEE Access, 2020, 8: 188485-188499.
- [4] HE Z, AI Q, LI Y. Artificial Intelligence in Logistics and Supply Chain Management: A Systematic Literature Review[J]. International Journal of Logistics Research and Applications, 2022, 25(11-12): 1285-1311.
- [5] NEWMAN S. Building Microservices: Designing Fine-Grained Systems[M]. Sebastopol: O'Reilly Media, 2015.
- [6] LEWIS J, FOWLER M. Microservices: a definition of this new architectural term[EB/OL]. (2014-03-25) [2025-05-04].
<https://martinfowler.com/articles/microservices.html>.
- [7] RICHARDSON C. Microservices Patterns: With examples in Java[M]. Shelter Island: Manning Publications, 2019.
- [8] ZIMMERMANN O. Microservices tenets[J]. Computer Science-Research and Development, 2017, 32(3-4): 301-310.
- [9] BALALAIE A, HEYDARNOORI A, JAMSHIDI P. Microservices architecture enables devops: Migration to a cloud-native architecture[J]. IEEE Software, 2016, 33(3): 42-52.
- [10] CONWAY M E. How do committees invent?[J]. Datamation, 1968, 14(4): 28-31.
- [11] DRAGONI N, GIALLORENZO S, LAFUENTE A L, et al. Microservices: yesterday, today, and tomorrow[M]//Present and Ulterior Software Engineering. Cham: Springer, 2017: 195-216.

- [12] VILLAMIZAR M, GARCÉS O, OCHOA L, et al. Cost comparison of running web applications in the cloud using monolithic, microservice, and AWS Lambda architectures[J]. *Service Oriented Computing and Applications*, 2017, 11(2): 233-247.
- [13] JOSUTTIS N M. *SOA in Practice: The Art of Distributed System Design*[M]. Sebastopol: O'Reilly Media, 2007.
- [14] SHARMA V, SINGH S. A Comparative Study of Message Queuing Systems: Kafka, RabbitMQ, and ActiveMQ[C]//2021 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS). Greater Noida: IEEE, 2021: 104-109.
- [15] KLEPPMANN M. *Designing Data-Intensive Applications*[M]. Sebastopol: O'Reilly Media, 2017.
- [16] 张亮, 屈钢, 吴 D. *大型网站技术架构:核心原理与案例分析*[M]. 北京: 电子工业出版社, 2013.
- [17] HUMBLE J, FARLEY D. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*[M]. Boston: Addison-Wesley Professional, 2010.
- [18] 陈皓. 消费者驱动的契约测试 (Consumer-Driven Contract Testing)[EB/OL]. (2016-09-08) [2025-05-04]. <https://coolshell.cn/articles/17670.html>.
- [19] 贾志刚, 王志刚, 李娜. 基于微服务架构的物流运输管理系统设计[J]. *软件导刊*, 2018, 17(11): 104-107.
- [20] 王珊珊, 孙力. 基于 SpringBoot 的物流管理系统设计与实现[J]. *电脑知识与技术*, 2021, 17(23): 88-90.
- [21] RODRIGUES F, RUIZ L, SICILIA M Á. API Gateways: Taxonomy, Survey, and Challenges[M]//*Service-Oriented Computing – ICSOC 2018 Workshops*. Cham: Springer, 2019: 171-183.
- [22] MICHAEL T N. *Release It!: Design and Deploy Production-Ready Software*[M]. 2nd ed. Raleigh: Pragmatic Bookshelf, 2018.
- [23] EVANS E. *Domain-Driven Design: Tackling Complexity in the Heart of*

- Software[M]. Boston: Addison-Wesley Professional, 2003.
- [24] 胡强. MySQL 数据库常见问题分析与研究[J]. 电脑编程技巧与维护, 2019(12): 91-92.
- [25] ARMS A, HLIBOWICKI A, NORMAN B, et al. R2DBC: Reactive Relational Database Connectivity[J]. Queue, 2021, 19(1): 48-69.
- [26] LUKAS M A, DE GOEDE R. Hands-On Reactive Programming in Spring 5[M]. Birmingham: Packt Publishing, 2018.
- [27] STEFANE M, LUEKE K. Reactive Spring[M]. Shelter Island: Manning Publications, 2021.
- [28] 邱书洋. Redis 缓存技术研究及应用[D]. 郑州: 郑州大学, 2016.
- [29] 杨洪. 基于 Redis 的共享缓存分析[J]. 数字技术与应用, 2024, 42(01): 110-112.
- [30] MERKEL D. Docker: lightweight linux containers for consistent development and deployment[J]. Linux Journal, 2014, 2014(239): 2.
- [31]

2 附录 A 数据模型对照表

模块	核心表/实体	主要字段	作用说明
认证与用户	user 、 shop_info 、 driver_info	id 、 username 、 secret 、 permission、 status	用于统一身份认证、角色区分及商户/司机审核状态维护。
顾客侧	customer、 address	id、 user_id、 name、 phone、 full_address 、 latitude 、 longitude	用于维护顾客基础信息及收货地址，为下单和末端配送提供输入。
商户与库存	shop、 goods、 warehouse、 stock	shop_id 、 goods_id 、 warehouse_id、 quantity、 status	用于维护商品、仓库与库存分布关系，支撑分仓发货和库存校验。
订单模块	order_info、 order_item	order_no 、 customer_id 、 shop_id 、 order_status 、 payment_status、 warehouse_id	用于记录订单主信息与订单明细，是履约链路的核心业务数据。
配送与路线	delivery_order 、 logistics_route	delivery_no 、 driver_id 、 route_no 、 segment_type 、 route_status、 planned_route	用于承载配送任务、干线与末端路线信息，以及运输过程状态。
中转站与网	logistics_hub 、 national_hub、 hub_link	hub_name 、 hub_level 、 capacity_daily 、 base_cost_per_unit	用于描述城市级中转站和全国干线网络拓扑，为调度与运力规划提供节点和边数据。

络			
调			
度	dispatch_pool	、 batch_no 、 batch_status 、	用于维护待调度订
与	logistics_batch	、 visit_sequence 、	单池、配送批次及
批	logistics_batch_item	stop_sequence、 item_status	批次内逐单停靠顺
次			序。
运			
力	flow_plan	、 plan_date 、 total_cost 、	用于保存全国最小
规	flow_plan_item	、 from_hub_id 、 to_hub_id 、	费用最大流计算结
划		allocated_flow	果及各干线边上的
			流量分配。

3 附录 B API 接口清单

服务	接口前缀	代表性接口	用途说明
auth-service	/api/auth	POST /register; POST /login; GET /me	完成用户注册、登录认证和当前用户信息获取。
customer-service	/api/customer	GET /profile; GET /address/list; POST /address	用于顾客信息维护和地址管理。
shop-service	/api/shop 、 /api/stock 、 /api/warehouse	GET /goods/list; POST /goods; GET /warehouse/list; PUT /stock	完成商品、仓库和库存的统一管理。
order-service	/api/order	POST /order; GET /order/list; PUT /order/{id}/ship; DELETE /order/{id}	负责订单创建、订单查询、商户发货和订单取消。
driver-	/api/driver、 /api/delivery	GET /delivery/hall ; POST	支撑司机

service		/delivery/{id}/accept ; POST 接单、配送 /delivery/{id}/arrive-hub 状态推进 和中转站 到达确认。 负责路径
logistics- service	/api/logistics	POST /routes ; GET 规划、物流 /routes/{id}/journey ; GET 轨迹查询、 /dispatch/preview ; POST 末端调度 /dispatch/execute 预览与批 次执行。 用于全国
national- network	/api/logistics/national- network /api/logistics/flow-plan	GET /hubs; GET /links; POST 枢纽网络 、/dispatch/national/plan ; GET 维护与运 /flow-plan/latest 力规划结 果查询。 作为系统 唯一入口， 负责令牌 校验、路径 鉴权与身 份透传。
gateway- service	/api/**	统一路由转发与 JWT 过滤

4 附录 C RabbitMQ 消息流全景表

消息主题/队列	生产者	消费者	业务触发点	作用说明
QUEUE_STOCK_DEDUCT	order- service	shop- service	订单创建成功 后	异步扣减库存，降低下单 接口同步压力，并通过死 信机制处理 异常。
QUEUE_DELIVERY_CREATE	order-	driver-	商户发	创建初始配

	service	service	货或订送单，将任务单进入投递到待接配送阶单大厅。 段
QUEUE_DISPATCH_POOL_IN	order-service	logistics-service	将订单写入订单分调度池，作为配中转末端调度与站后批次规划输入。
QUEUE_HUB_ARRIVAL	driver-service	logistics-service	更新批次与干线司路线状态，触机到达发中转节点中转站到站处理逻辑。
QUEUE_LAST_MILE_ACTIVATE	logistics-service	driver-service	激活末端配中转站送任务，创建确认到可供司机接站后单的末端配送单。
QUEUE_LOCATION_REPORT	driver-service	logistics-service	同步当前位司机上置到路线记报GPS录中，用于地理位置图轨迹与运输状态展示。
QUEUE_BATCH_FINISH	driver-service	order-service / logistics-service	批次全联动更新订单完成状态、批次状态和统计数据。
DLQ_*（死信队列）	各业务队 列失败消 息	管理员补 偿任务或 人工处理	消费失用于保留异 败或重常消息，保障 试超限异步链路的 后可追踪性和

补偿能力。

5 致谢

感谢我的导师牛志华老师，在毕业设计期间对我悉心指导，帮助我修改完成本论文。是您的包容与鼓励，帮助我在艰难时刻克服所有这些苦难。

感谢各位任课老师引导我走上计算机领域的专业道路。

感谢辅导员的关心与帮助，让我不惧失败。

感谢父母多年的养育与栽培。

感谢男朋友一路上的精神支持。

感谢朋友们的陪伴。

付苗

上海大学

2025 年 05 月 17 日

